

SOLIDWORKS API (.NET VBA C#) — Examples (28/50):

Get_Faces_Affected_by_Draft_Feature_Exam- Get_Labels_of_Datum_Origin_Example_VBNET

SOLIDWORKS API Documentation (converted from CHM)

Get Faces Affected by Draft Feature Example (VBA)

This example shows how to get the faces affected by a draft feature.

NOTE: In SOLIDWORKS Design, a feature is comprised of faces. The faces created by a feature can be retrieved using `IFeature::GetFaces`. However, certain types of features only affect existing faces and do not create any new faces. Examples of these types of features are draft and scale features. For these features, `IFeature::GetFaces` returns an empty array (no faces). This sample code shows how to use `IDraftFeatureData2::GetDraftedEntities` to get the faces affected by a draft feature.

```

'-----
' Preconditions:
' 1. Open a part or assembly with a draft feature.
' 2. If you opened an assembly, then fully resolve the assembly if necessary.
' 3. Select a draft feature.
'
' Postconditions:
' 1. Deselects the draft feature.
' 2. Selects the faces affected by the draft feature.
' 3. Gets the type of draft feature as defined in swDraftType_e.
' 4. Examine the Immediate window and graphics area.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim swFeat As SldWorks.Feature
    Dim swDraftFeat As SldWorks.DraftFeatureData2
    Dim vEnt As Variant
    Dim vEntArr As Variant
    Dim swEnt As SldWorks.Entity
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swDraftFeat = swFeat.GetDefinition

    ' Type as defined in swDraftType_e
    Debug.Print swDraftFeat.Type

    swModel.ClearSelection2 True

    ' No need to roll back to get affected faces
    vEntArr = swDraftFeat.GetDraftedEntities
    For Each vEnt In vEntArr
        Set swEnt = vEnt
        bRet = swEnt.Select4(True, swSelData)
    Next
End Sub

```

Get Faces Affected by Draft Feature Example (VB.NET)

This example shows how to get the faces affected by a draft feature.

NOTE: In SOLIDWORKS Design, a feature is comprised of faces. The faces created by a feature can be retrieved using `IFeature::GetFaces`. However, certain types of features only affect existing faces and do not create any new faces. Examples of these types of features are draft and scale features. For these features, `IFeature::GetFaces` returns an empty array (no faces). This sample code shows how to use `IDraftFeatureData2::GetDraftedEntities` to get the faces affected by a draft feature.

```
'-----
' Preconditions:
' 1. Open a part or assembly with a draft feature.
' 2. If you opened an assembly, then fully resolve the assembly if necessary.
' 3. Select a draft feature.
'
' Postconditions:
' 1. Deselects the draft feature.
' 2. Selects the faces affected by the draft feature.
' 3. Gets the type of draft feature as defined in swDraftFeature_e.
' 4. Examine the Immediate window and graphics area.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim swFeat As Feature
        Dim swDraftFeat As DraftFeatureData2
        Dim vEnt As Object
        Dim vEntArr As Object
        Dim swEnt As Entity
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swDraftFeat = swFeat.GetDefinition

        ' Type as defined in swDraftType_e
        Debug.Print(swDraftFeat.Type)

        swModel.ClearSelection2(True)

        ' No need to roll back to get affected faces
        vEntArr = swDraftFeat.GetDraftedEntities
        For Each vEnt In vEntArr
            swEnt = vEnt
            bRet = swEnt.Select4(True, swSelData)
        Next
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Faces Affected by Feature Example (C#)

This example shows how to get the faces affected by a feature. This example also highlights the edges on each affected face in blue.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Select a feature, such as a draft, that affects faces.
//
// Postconditions:
// 1. Highlights the edges of the faces in blue that are affected
//    by the selected feature.
// 2. Examine the graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace GetFaceEdgeCount_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        ModelDocExtension swModelExt;
        Feature swFeature;
        object[] vAffectedFaces;
        object[] vEdges;
        int nFaceCount;
        int nEdgeCount;
        int i;
        int j;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swModel.ClearSelection2(true);

            nFaceCount = swFeature.GetAffectedFaceCount();
            Debug.Print("Number of faces affected by selected feature = " + nFaceCount);
            vAffectedFaces = (object[])swFeature.GetAffectedFaces();

            for (i = 0; i <= (nFaceCount - 1); i++)
            {
                nEdgeCount = ((Face)vAffectedFaces[i]).GetEdgeCount();
                Debug.Print("    Number of edges on Face " + i + " = " + nEdgeCount);
                vEdges = (object[])((Face)vAffectedFaces[i]).GetEdges();

                for (j = 0; j <= (nEdgeCount - 1); j++)
                {
                    ((Edge)vEdges[j]).Display(2, 0, 0, 1, true);
                }
            }
        }

        public SldWorks swApp;
    }
}
```

Get Faces Affected by Feature Example (VBA)

This example shows how to get the faces affected by a feature. This example also highlights the edges on each affected face in blue.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Select a feature, such as a draft, that affects faces.  
'  
' Postconditions:  
' 1. Highlights the edges of the faces in blue that are affected  
'    by the selected feature.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swModelExt As SldWorks.ModelDocExtension  
Dim swFeature As SldWorks.Feature  
Dim vAffectedFaces As Variant  
Dim vEdges As Variant  
Dim nFaceCount As Long  
Dim nEdgeCount As Long  
Dim i As Long  
Dim j As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)  
    swModel.ClearSelection2 (True)  
  
    nFaceCount = swFeature.GetAffectedFaceCount  
    Debug.Print "Number of faces affected by selected feature = " & nFaceCount  
    vAffectedFaces = swFeature.GetAffectedFaces  
  
    For i = 0 To (nFaceCount - 1)  
        nEdgeCount = vAffectedFaces(i).GetEdgeCount  
        Debug.Print "    Number of edges on Face " & i; " = " & nEdgeCount  
        vEdges = vAffectedFaces(i).GetEdges  
  
        For j = 0 To (nEdgeCount - 1)  
            vEdges(j).Display 2, 0, 0, 1, True  
        Next j  
    Next i  
  
End Sub
```

Get Faces Affected by Feature Example (VB.NET)

This example shows how to get the faces affected by a feature. This example also highlights the edges on each affected face in blue.

```

'-----
' Preconditions:
' 1. Open a model document.
' 2. Select a feature, such as a draft, that affects faces.
'
' Postconditions:
' 1. Highlights the edges of the faces in blue that are affected
'    by the selected feature.
' 2. Examine the graphics area.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swModelExt As ModelDocExtension
    Dim swFeature As Feature
    Dim vAffectedFaces As Object
    Dim vEdges As Object
    Dim nFaceCount As Integer
    Dim nEdgeCount As Integer
    Dim i As Integer
    Dim j As Integer

    Sub main()

        swModel = swApp.ActiveDoc
        swModelExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swModel.ClearSelection2(True)

        nFaceCount = swFeature.GetAffectedFaceCount
        Debug.Print("Number of faces affected by selected feature = " & nFaceCount)
        vAffectedFaces = swFeature.GetAffectedFaces

        For i = 0 To (nFaceCount - 1)
            nEdgeCount = vAffectedFaces(i).GetEdgeCount
            Debug.Print("    Number of edges on Face " & i & " = " & nEdgeCount)
            vEdges = vAffectedFaces(i).GetEdges

            For j = 0 To (nEdgeCount - 1)
                vEdges(j).Display(2, 0, 0, 1, True)
            Next j
        Next i

    End Sub

    Public swApp As SldWorks

```

```
End Class
```

Get Faces Affected by Scale Feature Example (VBA)

This example shows how to get the faces affected by a scale feature in a multibody part.

NOTE: In SOLIDWORKS Design, a feature is a collection of faces. You can get these faces using `IFeature::GetFaces`. However, some features only affect existing faces and do not create any new faces. Examples of these types of features are draft and scale features. For these features, `IFeature::GetFaces` returns an empty array (i.e., no faces). To get the faces affected by a scale feature, first get the solid bodies to which the scale feature is applied. Each body is composed of a number of faces, which you can use to determine where these faces are located after the scale feature is applied to the body. An interesting issue is how to locate a face after a rebuild because a face might not exist after a rebuild. To address this issue, SOLIDWORKS has safe entities, which are entities that survive a rebuild (i.e., are valid across a rebuild).

```
'-----
' Preconditions:
' 1. Open a multibody part that has a scale feature.
' 2. Select the scale feature.
'
' Postconditions:
' 1. Deselects the scale feature.
' 2. Selects the faces affected by the scale feature.
' 3. Examine the graphics area.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim swFeat As SldWorks.Feature
    Dim swScaleFeat As SldWorks.ScaleFeatureData
    Dim vBody As Variant
    Dim vBodyArr As Variant
    Dim swBody As SldWorks.Body2
    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim swSafeEnt As SldWorks.Entity
    Dim vSafeEnt As Variant
    Dim swSafeEntColl As New Collection
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swScaleFeat = swFeat.GetDefinition

    bRet = swScaleFeat.AccessSelections(swModel, Nothing)
    vBodyArr = swScaleFeat.Bodies
    For Each vBody In vBodyArr
        Set swBody = vBody
        Set swFace = swBody.GetFirstFace
        Do While Not swFace Is Nothing
            ' Face reference is only valid in rolled-back
            ' state, so get a more persistent face reference
            Set swEnt = swFace
            Set swSafeEnt = swEnt.GetSafeEntity
            swSafeEntColl.Add swSafeEnt
            Set swFace = swFace.GetNextFace
        Loop
    Next

    swScaleFeat.ReleaseSelectionAccess

    For Each vSafeEnt In swSafeEntColl
        Set swSafeEnt = vSafeEnt
        bRet = swSafeEnt.Select4(True, swSelData)
    Next

End Sub
```

Get Faces Associated with Feature Example (C#)

This example shows how to eliminate multiple feature faces.

NOTE: In SOLIDWORKS Design, a face is the result of evaluating a feature. A face can be owned by several features. `IFeature::GetFaces` returns all faces owned by a feature. This is different from faces highlighted in the user interface when a feature is selected, because the user interface filters out multiple feature faces. This filter is for display purposes only. An application must use `IFace::GetFeature` to filter out multiple feature faces. This method returns only the oldest feature from a face; that is, the first owning feature in the FeatureManager design tree.

```
//-----
// Preconditions:
// 1. Open a part document.
// 2. Select a feature in the FeatureManager design
//    tree.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets the name of the feature and number
//    of faces.
// 2. Colors the faces of the feature blue.
//    NOTE: The faces are the same faces as if you
//    selected the feature in the user interface.
// 3. Examine the Immediate window and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetFaceCountCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            Feature swFeat = default(Feature);
            Feature swFaceFeat = default(Feature);
            object[] faceArr = null;
            double[] featColors = null;
            Face2 swFace = default(Face2);
            Entity swEnt = default(Entity);
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSelData = (SelectData)swSelMgr.CreateSelectData();

            Debug.Print("Feature = " + swFeat.Name + " [" + swFeat.GetTypeName() + "]");
            Debug.Print("  Face count = " + swFeat.GetFaceCount());

            swModel.ClearSelection2(true);

            featColors = (double[])swModel.MaterialPropertyValues;
            featColors[0] = 0;    //R
            featColors[1] = 0;    //G
            featColors[2] = 1;    //B

            faceArr = (object[])swFeat.GetFaces();
            if ((faceArr == null))
                return;
            foreach (object oneFace in faceArr)
            {
                swFace = (Face2)oneFace;
                swEnt = (Entity)swFace;
                swFaceFeat = (Feature)swFace.GetFeature();
                // Check to see if face is owned by multiple features
                if (object.ReferenceEquals(swFaceFeat, swFeat))
                {
                    status = swEnt.Select4(true, swSelData);
                    Debug.Assert(status);
                    swFace.MaterialPropertyValues = featColors;
                }
                else
                {
                    Debug.Print("  Other feature = " + swFaceFeat.Name + " [" + swFaceFeat.GetTypeName() + "]");
                }
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Get Faces Associated with Feature Example (VBA)

This example shows how to eliminate multiple feature faces.

NOTE: In SOLIDWORKS Design, a face is the result of evaluating a feature. A face can be owned by several features. IFeature::GetFaces returns all faces owned by a feature. This is different from faces highlighted in the user interface when a feature is selected, because the user interface filters out multiple feature faces. This filter is for display purposes only. An application must use IFace::GetFeature to filter out multiple feature faces. This method returns only the oldest feature from a face; that is, the first owning feature in the FeatureManager design tree.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Select a feature in the FeatureManager design
'   tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the name of the feature and number
'   of faces.
' 2. Colors the faces of the feature blue.
'   NOTE: The faces are the same faces as if
'   you selected the feature in the user interface.
' 3. Examine the Immediate window and graphics area.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim swFeat As SldWorks.Feature
    Dim swFaceFeat As SldWorks.Feature
    Dim faceArr As Variant
    Dim oneFace As Variant
    Dim featColors As Variant
    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim status As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swSelData = swSelMgr.CreateSelectData

    Debug.Print "Feature = " + swFeat.Name + " [" + swFeat.GetTypeName + "]"
    Debug.Print "  Face count = " & swFeat.GetFaceCount

    swModel.ClearSelection2 True

    featColors = swModel.MaterialPropertyValues
    featColors(0) = 0 'R
    featColors(1) = 0 'G
    featColors(2) = 1 'B
    faceArr = swFeat.GetFaces: If IsEmpty(faceArr) Then Exit Sub

    For Each oneFace In faceArr
        Set swFace = oneFace
        Set swEnt = swFace
        Set swFaceFeat = swFace.GetFeature

        ' Check to see if face is owned by multiple features
        If swFaceFeat Is swFeat Then
            status = swEnt.Select4(True, swSelData): Debug.Assert status
            swFace.MaterialPropertyValues = featColors
        Else
            Debug.Print "  Other feature = " & swFaceFeat.Name + " [" + swFaceFeat.GetTypeName + "]"
        End If
    Next

End Sub

```

Get Faces Associated with Feature Example (VB.NET)

This example shows how to eliminate multiple feature faces.

NOTE: In SOLIDWORKS Design, a face is the result of evaluating a feature. A face can be owned by several features. IFeature::GetFaces returns all faces owned by a feature. This is different from faces highlighted in the user interface when a feature is selected, because the user interface filters out multiple feature faces. This filter is for display purposes only. An application must use IFace::GetFeature to filter out multiple feature faces. This method returns only the oldest feature from a face; that is, the first owning feature in the FeatureManager design tree.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Select a feature in the FeatureManager design
'   tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the name of the feature and number
'   of faces.
' 2. Colors the faces of the feature blue.
'   NOTE: The faces are the same faces as if
'   you selected the feature in the user interface.
' 3. Examine the Immediate window and graphics area.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim swFeat As Feature
        Dim swFaceFeat As Feature
        Dim faceArr As Object
        Dim oneFace As Object
        Dim featColors As Object
        Dim swFace As Face2
        Dim swEnt As Entity
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swSelData = swSelMgr.CreateSelectData
        Debug.Print("Feature = " + swFeat.Name + " [" + swFeat.GetTypeName + "]")
        Debug.Print(" Face count = " & swFeat.GetFaceCount)
        swModel.ClearSelection2(True)
        featColors = swModel.MaterialPropertyValues
        featColors(0) = 0 'R
        featColors(1) = 0 'G
        featColors(2) = 1 'B
        faceArr = swFeat.GetFaces : If IsNothing(faceArr) Then Exit Sub
        For Each oneFace In faceArr
            swFace = oneFace
            swEnt = swFace
            swFaceFeat = swFace.GetFeature
            ' Check to see if face is owned by multiple features
            If swFaceFeat Is swFeat Then
                status = swEnt.Select4(True, swSelData) : Debug.Assert(status)
                swFace.MaterialPropertyValues = featColors
            Else
                Debug.Print(" Other feature = " & swFaceFeat.Name + " [" + swFaceFeat.GetTypeName + "]")
            End If
        Next

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Get Feature Type and Name Example (VBA)

This example shows how to get the feature type and name of the selected feature for use with IModelDocExtension::SelectByID2.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\floxpress\ball_valve\ball_valve.sldasm.  
' 2. Expand any component in the FeatureManager design tree  
'    and select one of its features.  
'  
' Postconditions:  
' 1. Gets the selected feature's type and name.  
' 2. Examine the Immediate window.  
'  
' NOTE: Because this assembly document is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim featName As String, featType As String  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
  
    ' Get the selected feature  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
  
    swModel.ClearSelection2 True  
  
    ' Get the feature's type and name  
    featName = swFeat.GetNameForSelection(featType)  
    swModelDocExt.SelectByID2 featName, featType, 0, 0, 0, True, 0, Nothing, 0  
  
    ' Print the feature's type and name  
    ' to the Immediate window  
    Debug.Print "Feature type: " & featType  
    Debug.Print "Feature name: " & featName  
  
End Sub
```

Get Feature Type and Name Example (VB.NET)

This example shows how to get the feature type and name of the selected feature for use with IModelDocExtension::SelectByID2.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\floypress\ball_valve\ball_valve.sldasm.
' 2. Expand any component in the FeatureManager design tree
'    and select one of its features.
'
' Postconditions:
' 1. Gets the selected feature's type and name.
' 2. Examine the Immediate window.
'
' NOTE: Because this assembly document is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension

        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim featName As String, featType As String

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swModelDocExt = swModel.Extension

        ' Get the selected feature
        swFeat = swSelMgr.GetSelectedObject6(1, -1)

        swModel.ClearSelection2(True)

        featType = ""
        featName = ""

        ' Get the feature's type and name
        featName = swFeat.GetNameForSelection(featType)
        swModelDocExt.SelectByID2(featName, featType, 0, 0, 0, True, 0, Nothing, 0)

        ' Print the feature's type and name
        ' to the Immediate window
        Debug.Print("Feature type: " & featType)
        Debug.Print("Feature name: " & featName)

    End Sub

    ''' <summary>
    ''' The Sldworks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Features in Reverse Order Example (C#)

This example shows how to get the names and types of features in the FeatureManager design tree in reverse chronological order.

```
//-----
// Preconditions:
// 1. Open a part.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the names and types of features
//    in the FeatureManager design tree
//    in reverse chronological order.
// 2. Examine the Immediate window and
//    FeatureManager design tree.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Runtime.InteropServices;
using System.Diagnostics;

namespace GetFeatureInReverseOrderCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Feature swFeat = default(Feature);
            int i = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("");
            Debug.Print("Feature Name and Type");
            Debug.Print("-----");
            for (i = 0; i <= swModel.GetFeatureCount() - 1; i++)
            {
                swFeat = (Feature)swModel.FeatureByPositionReverse(i);
                if ((swFeat != null))
                {
                    Debug.Print(swFeat.Name + " [" + swFeat.GetTypeName() + "]");
                }
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Get Features in Reverse Order Example (VBA)

This example shows how to get the names and types of features in the FeatureManager design tree in reverse chronological order.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the names and types of features  
'    in the FeatureManager design tree  
'    in reverse chronological order.  
' 2. Examine the Immediate window and  
'    FeatureManager design tree.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swFeat               As SldWorks.Feature  
    Dim i                   As Long  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print ""  
    Debug.Print "Feature Name and Type"  
    Debug.Print "=====  
    For i = 0 To swModel.GetFeatureCount - 1  
        Set swFeat = swModel.FeatureByPositionReverse(i)  
        If Not swFeat Is Nothing Then  
            Debug.Print swFeat.Name & " [" & swFeat.GetTypeName & "]"  
        End If  
    Next i  
  
End Sub
```

Get Features in Reverse Order Example (VB.NET)

This example shows how to get the names and types of features in the FeatureManager design tree in reverse chronological order.

```

-----
Preconditions:
1. Open a part.
2. Open the Immediate window.
Postconditions:
1. Gets the names and types of features
   in the FeatureManager design tree in
   reverse chronological order.
2. Examine the Immediate window and
   FeatureManager design tree.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Runtime.InteropServices
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swFeat As Feature
        Dim i As Integer

        swModel = swApp.ActiveDoc

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print("")
        Debug.Print("Feature Name and Type")
        Debug.Print("=====")
        For i = 0 To swModel.GetFeatureCount - 1
            swFeat = swModel.FeatureByPositionReverse(i)
            If Not swFeat Is Nothing Then
                Debug.Print(swFeat.Name & " [" & swFeat.GetTypeName & "]")
            End If
        Next i

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Features of Multibody Sheet Metal Part Example (C#)

This example shows how to sort a cut-list folder of a multibody sheet metal part.

```
// -----
// Preconditions:
// 1. Open C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\api\weldment_box3.sldprt.
// 2. Inspect the cut list folder.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets the number and names of the features in the cut list bodies.
// 2. Sets the cut list sorting options.
// 3. Sorts the cut list.
// 4. Inspect the sorted cut list folder in the Immediate window.
// -----
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Security;
using System.Text;
using System.Threading.Tasks;
using Microsoft.VisualBasic;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

using System.Windows;
using System.Windows.Forms;

namespace CutListSort_CSharp
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            Feature swFeat;
            BodyFolder swBodyFolder;
            SelectionMgr selMgr;
            Body2 swBody;
            object[] Bodies;
            object[] Features;
            CutListSortOptions CutListSortOptions;
            int i;
            int j;
            bool boolstatus;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            selMgr = (SelectionMgr)swModel.SelectionManager;

            boolstatus = swModel.Extension.SelectByID2("Solid Bodies", "BDYFOLDER", 0, 0, 0, false, 0, null/* TODO Change to default(_) if this is not
a reference type */, 0);
            swFeat = (Feature)selMgr.GetSelectedObject6(1, -1);

            swBodyFolder = (BodyFolder)swFeat.GetSpecificFeature2();
            swBodyFolder.SetAutomaticCutList(true);
            swBodyFolder.SetAutomaticUpdate(false);
            Bodies = (object[])swBodyFolder.GetBodies();
            Debug.Print("    Number of bodies: " + swBodyFolder.GetBodyCount());
            Debug.Print("    Cut list type: " + swBodyFolder.GetCutListType());
            Debug.Print("    Generate cut list automatically? " + swBodyFolder.GetAutomaticCutList());
            Debug.Print("    Automatically update cut list? " + swBodyFolder.GetAutomaticUpdate());
            for (i = 0; i <= (swBodyFolder.GetBodyCount() - 1); i++)
            {
                swBody = (Body2)Bodies[i];
                Features = (object[])swBody.GetFeatures();
                Debug.Print("    Number of features in body #" + i + 1 + ": " + swBody.GetFeatureCount());
                for (j = 0; j <= (swBody.GetFeatureCount() - 1); j++)
                {
                    Debug.Print("        Name of feature: " + ((Feature)Features[j]).GetTypeName2());
                }
            }

            // Sort the cut list
            CutListSortOptions = (CutListSortOptions)swBodyFolder.GetCutListSortOptions();
            CutListSortOptions.CollectIdenticalBodies = true;
            boolstatus = swBodyFolder.SortCutList(CutListSortOptions, false);
        }
        /// <summary>
        ///     ''' The SldWorks swApp variable is pre-assigned for you.
        ///     ''' </summary>
        public SldWorks swApp;
    }
}
```

Get Features of Multibody Sheet Metal Part Example (VBA)

This example shows how to sort the cut-list folder of a multibody sheet metal part.

```
'-----
' Preconditions:
' 1. Open C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\api\weldment_box3.sldprt.
' 2. Inspect the cut list folder.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the number and names of the features in the cut list bodies.
' 2. Sets the cut list sorting options.
' 3. Sorts the cut list.
' 4. Inspect the sorted cut list folder in the Immediate window.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeat As SldWorks.Feature
Dim swBodyFolder As SldWorks.BodyFolder
Dim selMgr As SldWorks.SelectionMgr
Dim swBody As Body2
Dim Bodies As Variant
Dim Features As Variant
Dim CutListSortOptions As SldWorks.CutListSortOptions
Dim i As Long
Dim j As Long
Dim boolstatus As Boolean

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set selMgr = swModel.SelectionManager

boolstatus = swModel.Extension.SelectByID("Solid Bodies", "BDYFOLDER", 0, 0, 0, False, 0, Nothing, 0)
Set swFeat = selMgr.GetSelectedObject6(1, -1)

Set swBodyFolder = swFeat.GetSpecificFeature2
swBodyFolder.SetAutomaticCutList True
swBodyFolder.SetAutomaticUpdate False
Bodies = swBodyFolder.GetBodies
Debug.Print " Number of bodies: " & swBodyFolder.GetBodyCount
Debug.Print " Cut list type: " & swBodyFolder.GetCutListType
Debug.Print " Generate cut list automatically? " & swBodyFolder.GetAutomaticCutList
Debug.Print " Automatically update cut list? " & swBodyFolder.GetAutomaticUpdate
For i = 0 To (swBodyFolder.GetBodyCount - 1)
    Set swBody = Bodies(i)
    Features = swBody.GetFeatures
    Debug.Print " Number of features in body #" & i + 1 & ": " & swBody.GetFeatureCount
    For j = 0 To (swBody.GetFeatureCount - 1)
        Debug.Print " Name of feature: " & Features(j).GetTypeName2
    Next j
Next i

' Sort the cut list
Set CutListSortOptions = swBodyFolder.GetCutListSortOptions
CutListSortOptions.CollectIdenticalBodies = True
boolstatus = swBodyFolder.SortCutList(CutListSortOptions, False)
```

End Sub

Get Features of Multibody Sheet Metal Part Example (VB.NET)

This example shows how to sort a cut-list folder of a multibody sheet metal part.

```

'-----
' Preconditions:
' 1. Open C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\api\weldment_box3.sldprt.
' 2. Inspect the cut list folder.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the number and names of the features in the cut list bodies.
' 2. Sets the cut list sorting options.
' 3. Sorts the cut list.
' 4. Inspect the sorted cut list folder in the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

Sub main()

    Dim swModel As ModelDoc2
    Dim swFeat As Feature
    Dim swBodyFolder As BodyFolder
    Dim selMgr As SelectionMgr
    Dim swBody As Body2
    Dim Bodies As Object
    Dim Features As Object
    Dim CutListSortOptions As CutListSortOptions
    Dim i As Integer
    Dim j As Integer
    Dim boolstatus As Boolean

    swModel = swApp.ActiveDoc
    selMgr = swModel.SelectionManager

    boolstatus = swModel.Extension.SelectByID2("Solid Bodies", "BDYFOLDER", 0, 0, 0, False, 0, Nothing, 0)
    swFeat = selMgr.GetSelectedObject6(1, -1)

    swBodyFolder = swFeat.GetSpecificFeature2
    swBodyFolder.SetAutomaticCutList(True)
    swBodyFolder.SetAutomaticUpdate(False)
    Bodies = swBodyFolder.GetBodies()
    Debug.Print(" Number of bodies: " & swBodyFolder.GetBodyCount())
    Debug.Print(" Cut list type: " & swBodyFolder.GetCutListType())
    Debug.Print(" Generate cut list automatically? " & swBodyFolder.GetAutomaticCutList())
    Debug.Print(" Automatically update cut list? " & swBodyFolder.GetAutomaticUpdate())
    For i = 0 To (swBodyFolder.GetBodyCount() - 1)
        swBody = Bodies(i)
        Features = swBody.GetFeatures()
        Debug.Print(" Number of features in body #" & i + 1 & ": " & swBody.GetFeatureCount())
        For j = 0 To (swBody.GetFeatureCount - 1)
            Debug.Print(" Name of feature: " & Features(j).GetTypeName2())
        Next j
    Next i

    ' Sort the cut list
    CutListSortOptions = swBodyFolder.GetCutListSortOptions
    CutListSortOptions.CollectIdenticalBodies = True
    boolstatus = swBodyFolder.SortCutList(CutListSortOptions, False)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get File Summary Information Example (VBA)

This example shows how to get the file summary information for an active SOLIDWORKS Design document.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the summary information for  
'    the model document.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    Debug.Print "File = " + swModel.GetPathName  
  
    Debug.Print " Title           = " + swModel.SummaryInfo(swSumInfoTitle)  
    Debug.Print " Subject          = " + swModel.SummaryInfo(swSumInfoSubject)  
    Debug.Print " Author           = " + swModel.SummaryInfo(swSumInfoAuthor)  
    Debug.Print " Keywords          = " + swModel.SummaryInfo(swSumInfoKeywords)  
    Debug.Print " Comment           = " + swModel.SummaryInfo(swSumInfoComment)  
    Debug.Print " Saved by          = " + swModel.SummaryInfo(swSumInfoSavedBy)  
    Debug.Print " Date created       = " + swModel.SummaryInfo(swSumInfoCreateDate)  
    Debug.Print " Date saved         = " + swModel.SummaryInfo(swSumInfoSaveDate)  
    Debug.Print " Date created       = " + swModel.SummaryInfo(swSumInfoCreateDate2)  
    Debug.Print " Date saved         = " + swModel.SummaryInfo(swSumInfoSaveDate2)  
  
End Sub
```

Get Fixed Face of Flat-Pattern Feature Example (C#)

This example shows how to get the fixed face of a Flat-Pattern feature.

```
//-----
// Preconditions: Open a sheet metal part and select a Flat-Pattern feature.
//
// Postconditions:
// 1. Inspect the graphics area to see that the fixed face
//    of the Flat-Pattern feature is selected.
// 2. Press F5 to run the macro to completion.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace FixedFace2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            FlatPatternFeatureData swFlatPatt = default(FlatPatternFeatureData);
            Face2 swFixedFace = default(Face2);
            Entity swFixedFaceEntity = default(Entity);
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);

            swFlatPatt = (FlatPatternFeatureData)swFeat.GetDefinition();
            bRet = swFlatPatt.AccessSelections(swModel, null);

            swFixedFace = (Face2)swFlatPatt.FixedFace2;
            swFixedFaceEntity = (Entity)swFixedFace;
            bRet = swFixedFaceEntity.Select4(true, null);

            System.Diagnostics.
Debugger.Break();
// Press F5

            swFlatPatt.ReleaseSelectionAccess();
        }

        public SldWorks swApp;
    }
}
```

Get Fixed Face of Flat-Pattern Feature Example (VBA)

This example shows how to get the fixed face of a Flat-Pattern feature.

```
'-----  
' Preconditions: Open a sheet metal part and select a Flat-Pattern feature.  
'-----  
' Postconditions:  
' 1. Inspect the graphics area to see that the fixed face  
'    of the Flat-Pattern feature is selected.  
' 2. Press F5 to run the macro to completion.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swSelMgr              As SldWorks.SelectionMgr  
    Dim swFeat               As SldWorks.Feature  
    Dim swFlatPatt           As SldWorks.FlatPatternFeatureData  
    Dim swFixedFace          As SldWorks.Face2  
    Dim selectData           As SldWorks.SelectData  
    Dim bRet                 As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
  
    Set swFlatPatt = swFeat.GetDefinition  
    bRet = swFlatPatt.AccessSelections(swModel, Nothing)  
  
    Set swFixedFace = swFlatPatt.FixedFace2  
    bRet = swFixedFace.Select4(True, selectData)  
  
    Stop  
    ' Press F5  
  
    swFlatPatt.ReleaseSelectionAccess  
  
End Sub
```

Get Fixed Face of Flat-Pattern Feature Example (VB.NET)

This example shows how to get the fixed face of a Flat-Pattern feature.

```
'-----  
' Preconditions: Open a sheet metal part and select a Flat-Pattern feature.  
'-----  
' Postconditions:  
' 1. Inspect the graphics area to see that the fixed face  
'    of the Flat-Pattern feature is selected.  
' 2. Press F5 to run the macro to completion.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim swFeat As Feature  
        Dim swFlatPatt As FlatPatternFeatureData  
        Dim swFixedFace As Face2  
        Dim swFixedFaceEntity As Entity  
        Dim bRet As Boolean  
  
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
        swFeat = swSelMgr.GetSelectedObject6(1, -1)  
  
        swFlatPatt = swFeat.GetDefinition  
        bRet = swFlatPatt.AccessSelections(swModel, Nothing)  
  
        swFixedFace = swFlatPatt.FixedFace2  
        swFixedFaceEntity = swFixedFace  
        bRet = swFixedFaceEntity.Select4(True, Nothing)  
  
        Stop  
        ' Press F5  
  
        swFlatPatt.ReleaseSelectionAccess()  
  
    End Sub  
  
    Public swApp As SldWorks
```

```
End Class
```

Get Fixed Face of Flatten-Bends Feature Example (C#)

This example shows how to get the fixed face of a flatten-bends feature.

```
//-----
// Preconditions:
// 1. Open a sheet metal part.
// 2. Select a Process-Bends feature in the FeatureManager design
//    tree.
//
// Postconditions:
// 1. Rolls the FeatureManager design tree back to the feature
//    that contains the fixed face of the Process-Bends feature.
// 2. Selects the fixed face of the Process-Bends feature.
// 3. Examine the FeatureManager design tree and graphics area,
//    then press F5.
// 4. Rolls the FeatureManager design tree forward.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetFixedFace_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            BendsFeatureData swBends = default(BendsFeatureData);
            Face2 swFace = default(Face2);
            Entity swEntity = default(Entity);
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swBends = (BendsFeatureData)swFeat.GetDefinition();

            // Roll back to access the Process-Bends feature data
            bRet = swBends.AccessSelections(swModel, null);

            swFace = (Face2)swBends.GetFixedFace();
            swEntity = (Entity)swFace;
            bRet = swEntity.Select4(false, null);

            System.Diagnostics.Debugger.Break();
            // Press F5

            // Cancel any changes made
            swBends.ReleaseSelectionAccess();

        }

        public SldWorks swApp;
    }
}
```

Get Fixed Face of Flatten-Bends Feature Example (VBA)

This example shows how to get the fixed face of a flatten-bends feature.

```
'-----  
' Preconditions:  
' 1. Open a sheet metal part.  
' 2. Select a Process-Bends feature in the FeatureManager design  
'    tree.  
'  
' Postconditions:  
' 1. Rolls the FeatureManager design tree back to the feature  
'    that contains the fixed face of the Process-Bends feature.  
' 2. Selects the fixed face of the Process-Bends feature.  
' 3. Examine the FeatureManager design tree and graphics area,  
'    then press F5.  
' 4. Rolls the FeatureManager design tree forward.  
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim swBends As SldWorks.BendsFeatureData  
Dim swFace As SldWorks.Face2  
Dim swEntity As SldWorks.Entity  
Dim bRet As Boolean
```

```
Set swApp = CreateObject("SldWorks.Application")  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
Set swBends = swFeat.GetDefinition
```

```
' Roll back to access the Process-Bends feature data  
bRet = swBends.AccessSelections(swModel, Nothing)  
  
Set swFace = swBends.GetFixedFace  
Set swEntity = swFace  
bRet = swEntity.Select4(False, Nothing)  
  
Stop  
' Press F5  
  
' Cancel any changes made  
swBends.ReleaseSelectionAccess
```

End Sub

Get Fixed Face of Flatten-Bends Feature Example (VB.NET)

This example shows how to get the fixed face of a flatten-bends feature.

```
'-----
' Preconditions:
' 1. Open a sheet metal part.
' 2. Select a Process-Bends feature in the FeatureManager design
'    tree.
'
' Postconditions:
' 1. Rolls the FeatureManager design tree back to the feature
'    that contains the fixed face of the Process-Bends feature.
' 2. Selects the fixed face of the Process-Bends feature.
' 3. Examine the FeatureManager design tree and graphics area,
'    then press F5.
' 4. Rolls the FeatureManager design tree forward.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
Sub main()
```

```
    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swFeat As Feature
    Dim swBends As BendsFeatureData
    Dim swFace As Face2
    Dim swEntity As Entity
    Dim bRet As Boolean
```

```
    swModel = swApp.ActiveDoc
    swSelMgr = swModel.SelectionManager
    swFeat = swSelMgr.GetSelectedObject6(1, -1)
    swBends = swFeat.GetDefinition
```

```
    ' Roll back to access the Process-Bends feature data
    bRet = swBends.AccessSelections(swModel, Nothing)
    swFace = swBends.GetFixedFace
    swEntity = swFace
    bRet = swEntity.Select4(False, Nothing)
```

```
    Stop
    ' Press F5
```

```
    ' Cancel any changes made
    swBends.ReleaseSelectionAccess()
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Get Fixed Face of Sheet Metal Part Example (VBA)

This example shows how to find and select the fixed face or edge of a sheet metal part.

```
'-----
' Preconditions:
' 1. Open a sheet metal part.
' 2. Select the sheet metal feature in the FeatureManager design
'    tree.
'
' Postconditions:
' 1. Rolls back the sheet metal feature and selects the fixed face
'    or edge, if either exists.
' 2. Rolls forward the sheet metal part.
'-----
```

Option Explicit

Sub main()

```
Dim swApp                As SldWorks.SldWorks
Dim swModel              As SldWorks.ModelDoc2
Dim swSelMgr             As SldWorks.SelectionMgr
Dim swSelData            As SldWorks.SelectData
Dim swFeat               As SldWorks.Feature
Dim swSheetMetal         As SldWorks.SheetMetalFeatureData
Dim swFixedFaceEnt       As SldWorks.Entity
Dim bRet                 As Boolean
```

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swSelData = swSelMgr.CreateSelectData
Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
Set swSheetMetal = swFeat.GetDefinition
```

```
' Roll back the model to get access to fixed face
bRet = swSheetMetal.AccessSelections(swModel, Nothing)
```

```
Set swFixedFaceEnt = swSheetMetal.FixedReference
```

```
If Not swFixedFaceEnt Is Nothing Then
    Debug.Print "Got fixed face or fixed edge."
    bRet = swFixedFaceEnt.Select4(False, swSelData)
```

```
Else
    ' Do not have to specify fixed face or fixed edge in
    ' user interface, so not finding either is a valid
    Debug.Print "No fixed face or fixed edge."
End If
```

```
Stop
'Press F5
```

```
' Cancel changes and roll back the model
swSheetMetal.ReleaseSelectionAccess
```

End Sub

Get Flat-Pattern Folder Feature Example (C#)

This example shows how to get the flat-pattern folder and its contents.

```
//-----
// Preconditions: Open a multibody sheet metal part that has a folder
// of flat-patterns.
//
// Postconditions: Inspect the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace GetFlatPatternFolderFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 myModel;
        FeatureManager featureMgr;
        Feature feat;
        object[] featArray;

        int i;

        public void Main()
        {
            myModel = (ModelDoc2)swApp.ActiveDoc;
            featureMgr = myModel.FeatureManager;
            FlatPatternFolder flatPatternFolder = default(FlatPatternFolder);
            flatPatternFolder = (FlatPatternFolder)featureMgr.GetFlatPatternFolder();
            feat = flatPatternFolder.GetFeature();
            Debug.Print("Flat-pattern folder name: " + feat.Name);
            Debug.Print(" Number of flat-pattern features in the folder: " + flatPatternFolder.GetFlatPatternCount());
            featArray = (object[])flatPatternFolder.GetFlatPatterns();
            for (i = featArray.GetLowerBound(0); i <= featArray.GetUpperBound(0); i++)
            {
                feat = (Feature)featArray[i];
                Debug.Print("    " + feat.Name);
            }
        }

        public SldWorks swApp;
    }
}
```

Get Flat-Pattern Folder Feature Example (VBA)

This example shows how to get the flat-pattern folder and its contents.

```
'-----  
' Preconditions: Open a multibody sheet metal part that has a folder  
' of flat-patterns.  
'  
' Postconditions: Inspect the Immediate window.  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim myModel As SldWorks.ModelDoc2  
Dim featureMgr As SldWorks.FeatureManager  
Dim feat As SldWorks.Feature  
Dim featArray As Variant  
Dim i As Long  
Option Explicit  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set myModel = swApp.ActiveDoc  
    Set featureMgr = myModel.FeatureManager  
    Dim flatPatternFolder As SldWorks.FlatPatternFolder  
    Set flatPatternFolder = featureMgr.GetFlatPatternFolder  
    Set feat = flatPatternFolder.GetFeature  
    Debug.Print "Flat-pattern folder name: " & feat.Name  
    Debug.Print "  Number of flat-pattern features in the folder: " & flatPatternFolder.GetFlatPatternCount  
    featArray = flatPatternFolder.GetFlatPatterns  
    For i = LBound(featArray) To UBound(featArray)  
        Set feat = featArray(i)  
        Debug.Print "    " & feat.Name  
    Next i  
  
End Sub
```

Get Flat-Pattern Folder Feature Example (VB.NET)

This example shows how to get the flat-pattern folder and its contents.

```
'-----
' Preconditions: Open a multibody sheet metal part that has a folder
' of flat-patterns.
'
' Postconditions: Inspect the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim myModel As ModelDoc2
    Dim featureMgr As FeatureManager
    Dim feat As Feature
    Dim featArray() As Object
    Dim i As Integer

    Sub main()

        myModel = swApp.ActiveDoc
        featureMgr = myModel.FeatureManager
        Dim flatPatternFolder As FlatPatternFolder
        flatPatternFolder = featureMgr.GetFlatPatternFolder
        feat = flatPatternFolder.GetFeature
        Debug.Print("Flat-pattern folder name: " & feat.Name)
        Debug.Print("  Number of flat-pattern features in the folder: " & flatPatternFolder.GetFlatPatternCount)
        featArray = flatPatternFolder.GetFlatPatterns
        For i = featArray.GetLowerBound(0) To featArray.GetUpperBound(0)
            feat = featArray(i)
            Debug.Print("    " & feat.Name)
        Next i
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Format of Note Text Example (VBA)

This example shows how to find out whether the font style for notes is italic.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets whether the text format for notes  
'    is italic.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swTextFormat As SldWorks.TextFormat  
Dim TextFormatObj As Object  
Dim retval As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set TextFormatObj = swModel.GetUserPreferenceTextFormat(swDetailingNoteTextFormat)  
    Set swTextFormat = TextFormatObj  
    retval = swTextFormat.Italic  
    Debug.Print retval  
  
End Sub
```

Get General Table Feature Example (C#)

This example shows how to get a general table feature and some of its table annotation data.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing document.
// 2. Inserts a table annotation.
// 3. Gets the general table feature.
// 4. Prints the name of the general table feature and
//    some of its annotation table data the Immediate window.
// 5. Examine the Immediate window.
//
// NOTE: Because the drawing document is used elsewhere, do not
// save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GeneralTableFeatureCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDrawing = default(DrawingDoc);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            TableAnnotation swTableAnnotation = default(TableAnnotation);
            GeneralTableFeature swGeneralTableFeature = default(GeneralTableFeature);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeature = default(Feature);
            int nbrTableAnnotations = 0;
            object[] tableAnnotations = null;
            int i = 0;
            bool anchorAttached = false;
            int anchorType = 0;
            int nbrColumns = 0;
            int nbrRows = 0;

            //Open drawing document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\assem20.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            //Insert table annotation
            swDrawing = (DrawingDoc)swModel;
            swTableAnnotation = (TableAnnotation)swDrawing.InsertTableAnnotation2(false, 0.027512345659767, 0.132124518483965, 1, "", 2, 2);
            if ((swTableAnnotation != null))
            {
                swTableAnnotation.BorderLineWeight = 0;
                swTableAnnotation.GridLineWeight = 0;
            }

            //Select and get general table feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("General Table1", "GENERALTABLEFEAT", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swGeneralTableFeature = (GeneralTableFeature)swSelectionMgr.GetSelectedObject6(1, -1);
            swFeature = (Feature)swGeneralTableFeature.GetFeature();
            Debug.Print("General table feature name: " + swFeature.Name);

            //Get general table feature's annotation data
            nbrTableAnnotations = swGeneralTableFeature.GetTableAnnotationCount();
            Debug.Print("Number of annotations = " + nbrTableAnnotations);
            tableAnnotations = (object[])swGeneralTableFeature.GetTableAnnotations();
            for (i = 0; i <= (nbrTableAnnotations - 1); i++)
            {
                swTableAnnotation = (TableAnnotation)tableAnnotations[i];
                anchorAttached = swTableAnnotation.Anchored;
                Debug.Print("Table anchored = " + anchorAttached);
                anchorType = swTableAnnotation.AnchorType;
                Debug.Print("Anchor type = " + anchorType);
                nbrColumns = swTableAnnotation.ColumnCount;
                Debug.Print("Number of columns = " + nbrColumns);
                nbrRows = swTableAnnotation.RowCount;
                Debug.Print("Number of rows = " + nbrRows);
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Get General Table Feature Example (VBA)

This example shows how to get a general table feature and some of its table annotation data.

```
'-----
' Preconditions:
' 1. Verify that the specified drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Inserts a table annotation.
' 3. Gets the general table feature.
' 4. Prints the name of the general table feature and
'    some of its annotation table data the Immediate window.
' 5. Examine the Immediate window.
'
' NOTE: Because the drawing document is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim swTableAnnotation As SldWorks.TableAnnotation
Dim swGeneralTableFeature As SldWorks.GeneralTableFeature
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeature As SldWorks.Feature
Dim nbrTableAnnotations As Long
Dim tableAnnotations As Variant
Dim i As Long
Dim anchorAttached As Boolean
Dim anchorType As Long
Dim nbrColumns As Long
Dim nbrRows As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open drawing document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem20.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Insert table annotation
    Set swDrawing = swModel
    Set swTableAnnotation = swDrawing.InsertTableAnnotation2(False, 2.75123456559767E-02, 0.132124518483965, 1, "", 2, 2)
    If Not swTableAnnotation Is Nothing Then
        swTableAnnotation.BorderLineWeight = 0
        swTableAnnotation.GridLineWeight = 0
    End If

    'Select and get general table feature
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("General Table1", "GENERALTABLEFEAT", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swGeneralTableFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swFeature = swGeneralTableFeature.GetFeature
    Debug.Print "General table feature name: " & swFeature.Name

    'Get general table feature's annotation data
    nbrTableAnnotations = swGeneralTableFeature.GetTableAnnotationCount
    Debug.Print "Number of annotations = " & nbrTableAnnotations
    tableAnnotations = swGeneralTableFeature.GetTableAnnotations
    For i = 0 To (nbrTableAnnotations - 1)
        Set swTableAnnotation = tableAnnotations(i)
        anchorAttached = swTableAnnotation.Anchored
        Debug.Print "Table anchored      = " & anchorAttached
        anchorType = swTableAnnotation.AnchorType
        Debug.Print "Anchor type      = " & anchorType
        nbrColumns = swTableAnnotation.ColumnCount
        Debug.Print "Number of columns = " & nbrColumns
        nbrRows = swTableAnnotation.RowCount
        Debug.Print "Number of rows    = " & nbrRows
    Next i
End Sub
```

Get General Table Feature Example (VB.NET)

This example shows how to get a general table feature and some of its table annotation data.

```

-----
Preconditions:
1. Verify that the specified drawing document to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified drawing document.
2. Inserts a table annotation.
3. Gets the general table feature.
4. Prints the name of the general table feature and
   some of its annotation table data the Immediate window.
5. Examine the Immediate window.
NOTE: Because the drawing document is used elsewhere, do not
save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swDrawing As DrawingDoc
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim swTableAnnotation As TableAnnotation
        Dim swGeneralTableFeature As GeneralTableFeature
        Dim swSelectionMgr As SelectionMgr
        Dim swModelDocExt As ModelDocExtension
        Dim swFeature As Feature
        Dim nbrTableAnnotations As Integer
        Dim tableAnnotations() As Object
        Dim i As Integer
        Dim anchorAttached As Boolean
        Dim anchorType As Integer
        Dim nbrColumns As Integer
        Dim nbrRows As Integer

        'Open drawing document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\assem20.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Insert table annotation
        swDrawing = swModel
        swTableAnnotation = swDrawing.InsertTableAnnotation2(False, 0.0275123456559767, 0.132124518483965, 1, "", 2, 2)
        If Not swTableAnnotation.Is Nothing Then
            swTableAnnotation.BorderLineWeight = 0
            swTableAnnotation.GridLineWeight = 0
        End If

        'Select and get general table feature
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("General Table1", "GENERALTABLEFEAT", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swGeneralTableFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swFeature = swGeneralTableFeature.GetFeature
        Debug.Print("General table feature name: " & swFeature.Name)

        'Get general table feature's annotation data
        nbrTableAnnotations = swGeneralTableFeature.GetTableAnnotationCount
        Debug.Print("Number of annotations = " & nbrTableAnnotations)
        tableAnnotations = swGeneralTableFeature.GetTableAnnotations
        For i = 0 To (nbrTableAnnotations - 1)
            swTableAnnotation = tableAnnotations(i)
            anchorAttached = swTableAnnotation.Anchored
            Debug.Print("Table anchored = " & anchorAttached)
            anchorType = swTableAnnotation.AnchorType
            Debug.Print("Anchor type = " & anchorType)
            nbrColumns = swTableAnnotation.ColumnCount
            Debug.Print("Number of columns = " & nbrColumns)
            nbrRows = swTableAnnotation.RowCount
            Debug.Print("Number of rows = " & nbrRows)
        Next i

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get General Table Annotation Example (C#)

This example shows how to create a general table annotation for 3D PDF files in SOLIDWORKS MBD.

```
//-----  
// Preconditions: Open a part.  
//  
// Postconditions: None.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
namespace GetGeneralTableAnno_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 Part;  
        ModelDocExtension modelDocExt;  
  
        TableAnnotation myTable;  
  
        public void Main()  
        {  
            Part = (ModelDoc2)swApp.ActiveDoc;  
            modelDocExt = Part.Extension;  
            myTable = modelDocExt.GetGeneralTableAnnotation(true, 0, 0,  
(int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, "", 2, 2);  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Get General Table Annotation Example (VBA)

This example shows how to create a general table annotation for 3D PDF files in SOLIDWORKS MBD.

```
'-----  
' Preconditions: Open a part.  
'  
' Postconditions: None.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim modelDocExt As SldWorks.ModelDocExtension  
Dim myTable As SldWorks.TableAnnotation  
  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set modelDocExt = Part.Extension  
    Set myTable = modelDocExt.GetGeneralTableAnnotation(True, 0, 0, swBOMConfigurationAnchor_TopLeft, "", 2, 2)  
  
End Sub
```

Get General Table Annotation Example (VB.NET)

This example shows how to create a general table annotation for 3D PDF files in SOLIDWORKS MBD.

```
'-----  
' Preconditions: Open a part.  
' Postconditions: None.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Dim Part As ModelDoc2  
    Dim modelDocExt As ModelDocExtension  
    Dim myTable As TableAnnotation  
  
    Sub main()  
  
        Part = swApp.ActiveDoc  
        modelDocExt = Part.Extension  
        myTable = modelDocExt.GetGeneralTableAnnotation(True, 0, 0,  
swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, "", 2, 2)  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```

Get Gtol Frame Information Example (C#)

This example shows how to obtain information from a Gtol frame.

```
// =====  
// Preconditions:  
// 1. Open a drawing that contains a GTol created with either SolidWorks 2021 or SolidWorks 2022/2022+.  
// 2. Open an Immediate window.  
//  
// Postconditions:  
// 1. Select a Gtol in the graphics area.  
// A Geometric Tolerance PropertyManager opens.  
// 2. Click on the macro window and press F5.  
// 3. Gtol format type and schema version print to the Immediate window.  
// 4. Press F5.  
// 5. The frame XML schema string prints to the Immediate window.  
// 6. Press F5.  
// 7. If the selected Gtol can be converted from 2021, it is attempted.  
// 8. Inspect the Immediate window.  
// 9. Press F5.  
// 10. Gets and sets From and To text and toggles SEPARATE REQUIREMENT.  
// 11. Inspect the Immediate window.  
// 12. Press F5.  
// 13. Gtol frames added and deleted.  
// 14. Inspect the Immediate window.  
// 15. Press F5.  
// 16. Add Indicator tests.  
// 17. Inspect the Immediate window.  
// 18. Press F5.  
// 19. Get Indicator tests.  
// 20. Inspect the Immediate window.  
// 21. Press F5.  
// 22. Delete Indicator tests.  
// 23. Inspect the Immediate window.  
// 24. Press F5.  
// 25. Miscellaneous Indicator tests.  
// 26. Inspect the Immediate window.  
// 27. Press F5.  
// 28. Tolerance type tests.  
// 29. Inspect the Immediate window.  
// 30. Press F5.  
// 31. Get and set symbol XML string.  
// 32. Inspect the Immediate window.  
// =====
```

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows;  
using System.Windows.Forms;  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System.Diagnostics;  
using Microsoft.VisualBasic;  
  
namespace GetGtolFrameInfo_CSharp  
{  
    public partial class SolidWorksMacro  
    {  
        private ModelDoc2 swModel;  
        private bool boolstatus;  
        private int longstatus;
```

```

private int longwarnings;
private SelectionMgr swSelMgr;
private Gtol swGtol;
private GtolFrame swGtolFrame;
private GtolFrame swGtolFrame1;
private int FrameCount;
private string sXMLString;
private int iIndicator;
private int iFrame;
private int lFormatType;
private int lFormatType1;
private int lSchemaVersion;
private string sSchemaString;
private swGtolFormatConversionError_e swConversionError;
private string swFromText;
private string swToText;
private string swFromText1;
private string swToText1;
private int IndicatorCount;
private string sDatum;
private int lBorderType;
private int lTolType;
private int swToleranceType;
private int swTolType;
private int pos;

public void Main()
{

    swModel = (ModelDoc2)swApp.ActiveDoc;
    swSelMgr = (SelectionMgr)swModel.SelectionManager;

    System.Diagnostics.Debugger.Break(); // in the graphics area select Gtol created with either SolidWorks 2021 or
SolidWorks 2022/2022+

    Debug.Print("Select object type as defined by swSelectType_e: " + swSelMgr.GetSelectedObjectType3(1, -1));
    swGtol = (Gtol)swSelMgr.GetSelectedObject6(1, -1);

    swApp.GetGtolFormatData(out lFormatType, out lSchemaVersion);
    Debug.Print("Gtol format type as defined by swGtolFormatType_e: " + lFormatType);
    Debug.Print("Gtol schema version as defined by swGtolFormatSchemaVersion_e: " + lSchemaVersion);

    System.Diagnostics.Debugger.Break();

    sSchemaString = swApp.GetGtolFrameXMLSchema();
    Debug.Print("Gtol frame XML schema: " + sSchemaString);

    System.Diagnostics.Debugger.Break();

    boolstatus = swGtol.CanConvertFormat();
    if (boolstatus != false)
    {
        lFormatType1 = swGtol.GetFormat();
        if (lFormatType1 == (int)swGtolFormatType_e.GTOL_SW2021)
        {
            boolstatus = swModel.GetSaveFlag();
            if (boolstatus != false)
            {
                swModel.Save();
                Debug.Print("SaveFlag BEFORE ConvertFormat is called " + swModel.GetSaveFlag());
                swConversionError = (swGtolFormatConversionError_e)swGtol.ConvertFormat();
                boolstatus = swModel.GetSaveFlag(); // if successful, converted Gtol document should be dirty and return True
                Debug.Print("SaveFlag AFTER ConvertFormat is called " + boolstatus);
                lFormatType1 = swGtol.GetFormat(); // converted from 2021 to 2022
            }
        }
    }
}

```

```

System.Diagnostics.Debugger.Break();

boolstatus = swGtol.GetFromTo();
boolstatus = swGtol.GetFromToText(out swFromText, out swToText);

swFromText = "FromMe";
swToText = "ToYou";
boolstatus = swGtol.SetFromToText(true, swFromText, swToText);
boolstatus = swGtol.GetFromTo();

swFromText1 = "";

swToText1 = "";
boolstatus = swGtol.GetFromToText(out swFromText1, out swToText1); // should be "FromMe" and "ToMe"

Debug.Print("From text: " + swFromText1);
Debug.Print("To text: " + swToText1);

boolstatus = swGtol.SeparateRequirement;
if (boolstatus != false)
    swGtol.SeparateRequirement = false;
else
    swGtol.SeparateRequirement = true;

Debug.Print("SEPARATE REQUIREMENT ? " + swGtol.SeparateRequirement);

System.Diagnostics.Debugger.Break();

FrameCount = swGtol.GetFrameCount();
Debug.Print("Frame count: " + FrameCount);
boolstatus = swGtol.AddFrame();
FrameCount = swGtol.GetFrameCount(); // frame count should increase by one
Debug.Print("Frame added. Frame count now: " + FrameCount);
boolstatus = swGtol.DeleteFrame(FrameCount);
FrameCount = swGtol.GetFrameCount(); // frame count should decrease by 1, as just added frame was deleted
Debug.Print("Frame deleted. Frame count now: " + FrameCount);
boolstatus = swGtol.AddFrame();
FrameCount = swGtol.GetFrameCount(); // frame count should increase by 1
Debug.Print("Frame added. Frame count now: " + FrameCount);

System.Diagnostics.Debugger.Break();

swGtolFrame = (GtolFrame)swGtol.GetFrame(FrameCount);

IndicatorCount = swGtolFrame.GetIndicatorCount();

sDatum = "A";

Debug.Print(" ");
Debug.Print("Frame " + FrameCount + " IndicatorCount is " + IndicatorCount + ": AddIndicator test");

// supported border types: swGtolIndicatorBorderType_CollectionPlane, swGtolIndicatorBorderType_OrientationPlane
swGtolIndicatorBorderType_IntersectionPlane swGtolIndicatorBorderType_DirectionFeature
// supported tolerance types: swGcsPARALLEL swGcsPERP swGcsANG swGcsCIRCRUNOUT swGcsSYMMETRY

boolstatus = swGtolFrame.AddIndicator((int)swGtolIndicatorBorderType_e.swGtolIndicatorBorderType_IntersectionPlane,
(int)swGtolGeomCharSymbol_e.swGcsFLAT, sDatum); // unsupported TolType
Debug.Print("Unsupported TolType swGcsFLAT. AddIndicator return value is " + boolstatus);
boolstatus = swGtolFrame.AddIndicator((int)swGtolGeomCharSymbol_e.swGcsFLAT, (int)swGtolGeomCharSymbol_e.swGcsSYMMETRY,
sDatum); // unsupported BorderType
Debug.Print("Unsupported BorderType swGcsFLAT. AddIndicator return value is " + boolstatus);
boolstatus = swGtolFrame.AddIndicator((int)swGtolIndicatorBorderType_e.swGtolIndicatorBorderType_IntersectionPlane,
(int)swGtolGeomCharSymbol_e.swGcsSYMMETRY, sDatum); // supported BorderType and TolType

```

```

        Debug.Print("Supported BorderType swGtolIndicatorBorderType_IntersectionPlane and TolType swGcsSYMMETRY. AddIndicator
return value is " + boolstatus);

        IndicatorCount = swGtolFrame.GetIndicatorCount();
        Debug.Print("");
        Debug.Print("Frame " + FrameCount + " IndicatorCount after one successful AddIndicator is " + IndicatorCount); //
should increase by 1 from AddIndicator success
        Debug.Print(" ");

        System.Diagnostics.Debugger.Break();

        IndicatorCount = swGtolFrame.GetIndicatorCount();
        iIndicator = 0;
        iFrame = 0;
        FrameCount = swGtol.GetFrameCount();
        Debug.Print("FrameCount is " + FrameCount + ": GetIndicator test");
        for (iFrame = 1; iFrame <= FrameCount; iFrame++)
        {
            //lBorderType = -1;
            //lTolType = -1;
            sDatum = "";

            swGtolFrame1 = (GtolFrame)swGtol.GetFrame(iFrame);
            for (iIndicator = 1; iIndicator <= IndicatorCount; iIndicator++)
            {
                boolstatus = swGtolFrame1.GetIndicator(iIndicator, out lBorderType, out lTolType, out sDatum);
                Debug.Print("Frame: " + iFrame + " Indicator index: " + iIndicator + " (BorderType as defined by
swGtolIndicatorBorderType_e = " + lBorderType + " " + "TolType as defined by swGtolGeomCharSymbol_e = " + lTolType + " " + "Datum
= " + sDatum + ")");
            }
        }

        Debug.Print(" ");

        System.Diagnostics.Debugger.Break();

        Debug.Print("Frame " + FrameCount + " DeleteIndicator test");
        boolstatus = swGtolFrame.DeleteIndicator(IndicatorCount);
        Debug.Print("DeleteIndicator return value is " + boolstatus);
        IndicatorCount = swGtolFrame.GetIndicatorCount(); // should decrease by 1, as indicator deleted
        Debug.Print("IndicatorCount is " + IndicatorCount + " after DeleteIndicator");

        System.Diagnostics.Debugger.Break();

        Debug.Print(" ");
        Debug.Print("Miscellaneous Indicator test:");
        boolstatus = swGtolFrame.AddIndicator((int)swGtolIndicatorBorderType_e.swGtolIndicatorBorderType_OrientationPlane,
(int)swGtolGeomCharSymbol_e.swGcsANG, sDatum); // add indicator for different BorderType and TolType
        Debug.Print("Supported swGtolIndicatorBorderType_OrientationPlane BorderType AddIndicator return value is " +
boolstatus);
        IndicatorCount = swGtolFrame.GetIndicatorCount(); // should increase by 1
        Debug.Print("Frame " + FrameCount + " IndicatorCount after one successful AddIndicator is " + IndicatorCount); //
should increase by 1
        boolstatus = swGtolFrame.GetIndicator(IndicatorCount, out lBorderType, out lTolType, out sDatum);
        Debug.Print("Frame " + FrameCount + " just added GetIndicator: " + " (BorderType = " + lBorderType + " " + "TolType = "
+ lTolType + " " + "Datum = " + sDatum + ")");

        if (sDatum != "B")
            sDatum = "B";
        else
            sDatum = "A";

        boolstatus = swGtolFrame.SetIndicator(IndicatorCount,
(int)swGtolIndicatorBorderType_e.swGtolIndicatorBorderType_IntersectionPlane, (int)swGtolGeomCharSymbol_e.swGcsPERP, sDatum);
        Debug.Print("Indicator " + IndicatorCount + " SetIndicator return value is " + boolstatus);
        boolstatus = swGtolFrame.GetIndicator(IndicatorCount, out lBorderType, out lTolType, out sDatum);
        Debug.Print("Indicator " + IndicatorCount + " SetIndicator values " + " (BorderType = " + lBorderType + " " + "TolType
= " + lTolType + " " + "Datum = " + sDatum + ")");

```

```

        Debug.Print(" ");

        System.Diagnostics.Debugger.Break();

        swGtolFrame = (GtolFrame)swGtol.GetFrame(swGtol.GetFrameCount());
        Debug.Print("Frame " + 2 + " ToleranceType test");

        boolstatus = swGtolFrame.GetFrameToleranceType(out swToleranceType);
        Debug.Print("GetFrameToleranceType " + "returned " + boolstatus);
        Debug.Print("GetFrameToleranceType " + "before set returned tolerance type as defined by swGtolGeomCharSymbol_e: " +
swToleranceType);
        boolstatus = swGtolFrame.SetFrameToleranceType((int)swGtolGeomCharSymbol_e.swGcsROUND);
        Debug.Print("SetFrameToleranceType " + "returned " + boolstatus);
        boolstatus = swGtolFrame.GetFrameToleranceType(out swTolType);
        Debug.Print("GetFrameToleranceType " + "returned " + boolstatus);
        Debug.Print("GetFrameToleranceType " + "after set returned tolerance type as defined by swGtolGeomCharSymbol_e: " +
swTolType);
        Debug.Print("");

        System.Diagnostics.Debugger.Break();

        sXMLString = swGtolFrame.GetSymbolXml();
        Debug.Print("Symbol XML string: " + sXMLString);

        // Set a new symbol XML string of last added frame
        pos = -1;
        pos = Strings.InStr(1, sXMLString, ".01"); // Note: converted Gtol has tolerance format of .01, not 0.01
        if (pos != -1)
            sXMLString = Strings.Replace(sXMLString, ".01", ".02");
        else
        {
            pos = Strings.InStr(1, sXMLString, ".02");
            if (pos != -1)
                sXMLString = Strings.Replace(sXMLString, ".02", ".01");
        }

        boolstatus = swGtolFrame.SetSymbolXml(sXMLString);
        Debug.Print("New symbol XML string: " + sXMLString);

    }

    // The SldWorks swApp variable is pre-assigned for you.
    public SldWorks swApp;

}
}

```

Get Gtol Frame Information Example (VB.NET)

This example shows how to obtain information from a Gtol frame.

'=====

'Preconditions:

- '1. Open a drawing that contains a GTol created with either SolidWorks 2021 or SolidWorks 2022/2022+.
- '2. Open an Immediate window.

'

'Postconditions:

- '1. Select a Gtol in the graphics area.
' A Geometric Tolerance PropertyManager opens.
- '2. Click on the macro window and press F5.
- '3. Gtol format type and schema version print to the Immediate window.
- '4. Press F5.
- '5. The frame XML schema string prints to the Immediate window.
- '6. Press F5.
- '7. If the selected Gtol can be converted from 2021, it is attempted.
- '8. Inspect the Immediate window.
- '9. Press F5.
- '10. Gets and sets From and To text and toggles SEPARATE REQUIREMENT.
- '11. Inspect the Immediate window.
- '12. Press F5.
- '13. Gtol frames added and deleted.
- '14. Inspect the Immediate window.
- '15. Press F5.
- '16. Add Indicator tests.
- '17. Inspect the Immediate window.
- '18. Press F5.
- '19. Get Indicator tests.
- '20. Inspect the Immediate window.
- '21. Press F5.
- '22. Delete Indicator tests.
- '23. Inspect the Immediate window.
- '24. Press F5.
- '25. Miscellaneous Indicator tests.
- '26. Inspect the Immediate window.
- '27. Press F5.
- '28. Tolerance type tests.
- '29. Inspect the Immediate window.
- '30. Press F5.
- '31. Get and set symbol XML string.
- '32. Inspect the Immediate window.

'=====

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

Partial Class SolidWorksMacro

```
    Dim swModel As ModelDoc2
    Dim boolstatus As Boolean
    Dim longstatus As Integer, longwarnings As Integer
    Dim swSelMgr As SelectionMgr
    Dim swGtol As Gtol
    Dim swGtolFrame As GtolFrame
    Dim swGtolFrame1 As GtolFrame
    Dim FrameCount As Integer
    Dim sXMLString As String
    Dim iIndicator As Integer
    Dim iFrame As Integer
    Dim lFormatType As Integer
    Dim lFormatType1 As Integer
```

```

Dim lSchemaVersion As Integer
Dim sSchemaString As String
Dim swConversionError As swGtolFormatConversionError_e
Dim swFromText As String
Dim swToText As String
Dim swFromText1 As String
Dim swToText1 As String
Dim IndicatorCount As Integer
Dim sDatum As String
Dim lBorderType As swGtolIndicatorBorderType_e
Dim lTolType As swGtolGeomCharSymbol_e
Dim swToleranceType As swGtolGeomCharSymbol_e
Dim swTolType As Integer
Dim pos As Integer

Public Sub main()

    swModel = swApp.ActiveDoc
    swSelMgr = swModel.SelectionManager

    Stop 'in the graphics area select Gtol created with either SolidWorks 2021 or SolidWorks 2022/2022+

    Debug.Print("Select object type as defined by swSelectType_e: " & swSelMgr.GetSelectedObjectType3(1, -1))
    swGtol = swSelMgr.GetSelectedObject6(1, -1)

    swApp.GetGtolFormatData(lFormatType, lSchemaVersion)
    Debug.Print("Gtol format type as defined by swGtolFormatType_e: " & lFormatType)
    Debug.Print("Gtol schema version as defined by swGtolFormatSchemaVersion_e: " & lSchemaVersion)

    Stop

    sSchemaString = swApp.GetGtolFrameXMLSchema
    Debug.Print("Gtol frame XML schema: " & sSchemaString)

    Stop

    boolstatus = swGtol.CanConvertFormat
    If Not boolstatus = 0 Then

        lFormatType1 = swGtol.GetFormat
        If lFormatType1 = swGtol.FormatType_e.GTOL_SW2021 Then

            boolstatus = swModel.GetSaveFlag
            If boolstatus <> 0 Then
                swModel.Save()
            End If
            Debug.Print("SaveFlag BEFORE ConvertFormat is called " & swModel.GetSaveFlag)
            swConversionError = swGtol.ConvertFormat
            boolstatus = swModel.GetSaveFlag 'if successful, converted Gtol document should be dirty and return True
            Debug.Print("SaveFlag AFTER ConvertFormat is called " & boolstatus)
            lFormatType1 = swGtol.GetFormat 'converted from 2021 to 2022
        End If
    End If

    Stop

    boolstatus = swGtol.GetFromTo
    boolstatus = swGtol.GetFromToText(swFromText, swToText)

    swFromText = "FromMe"
    swToText = "ToYou"
    boolstatus = swGtol.SetFromToText(True, swFromText, swToText)
    boolstatus = swGtol.GetFromTo

```

```

swFromText1 = ""

swToText1 = ""
boolstatus = swGtol.GetFromToText(swFromText1, swToText1) 'should be "FromMe" and "ToMe"

Debug.Print("From text: " & swFromText1)
Debug.Print("To text: " & swToText1)

boolstatus = swGtol.SeparateRequirement
If Not boolstatus = 0 Then
    swGtol.SeparateRequirement = False
Else
    swGtol.SeparateRequirement = True
End If

Debug.Print("SEPARATE REQUIREMENT ? " & swGtol.SeparateRequirement)

Stop

FrameCount = swGtol.GetFrameCount
Debug.Print("Frame count: " & FrameCount)
boolstatus = swGtol.AddFrame
FrameCount = swGtol.GetFrameCount 'frame count should increase by one
Debug.Print("Frame added. Frame count now: " & FrameCount)
boolstatus = swGtol.DeleteFrame(FrameCount)
FrameCount = swGtol.GetFrameCount 'frame count should decrease by 1, as just added frame was deleted
Debug.Print("Frame deleted. Frame count now: " & FrameCount)
boolstatus = swGtol.AddFrame
FrameCount = swGtol.GetFrameCount 'frame count should increase by 1
Debug.Print("Frame added. Frame count now: " & FrameCount)

Stop

swGtolFrame = swGtol.GetFrame(FrameCount)

IndicatorCount = swGtolFrame.GetIndicatorCount

sDatum = "A"

Debug.Print(" ")
Debug.Print("Frame " & FrameCount & " IndicatorCount is " & IndicatorCount & ": AddIndicator test")

'supported border types: swGtolIndicatorBorderType_CollectionPlane, swGtolIndicatorBorderType_OrientationPlane
swGtolIndicatorBorderType_IntersectionPlane swGtolIndicatorBorderType_DirectionFeature
'supported tolerance types: swGcsPARALLEL swGcsPERP swGcsANG swGcsCIRCRUNOUT swGcsSYMMETRY

boolstatus = swGtolFrame.AddIndicator(swGtolIndicatorBorderType_e.swGtolIndicatorBorderType_IntersectionPlane,
swGtolGeomCharSymbol_e.swGcsFLAT, sDatum) 'unsupported TolType
Debug.Print("Unsupported TolType swGcsFLAT. AddIndicator return value is " & boolstatus)
boolstatus = swGtolFrame.AddIndicator(swGtolGeomCharSymbol_e.swGcsFLAT, swGtolGeomCharSymbol_e.swGcsSYMMETRY, sDatum)
'unsupported BorderType
Debug.Print("Unsupported BorderType swGcsFLAT. AddIndicator return value is " & boolstatus)
boolstatus = swGtolFrame.AddIndicator(swGtolIndicatorBorderType_e.swGtolIndicatorBorderType_IntersectionPlane,
swGtolGeomCharSymbol_e.swGcsSYMMETRY, sDatum) 'supported BorderType and TolType
Debug.Print("Supported BorderType swGtolIndicatorBorderType_IntersectionPlane and TolType swGcsSYMMETRY. AddIndicator
return value is " & boolstatus)

IndicatorCount = swGtolFrame.GetIndicatorCount
Debug.Print("")
Debug.Print("Frame " & FrameCount & " IndicatorCount after one successful AddIndicator is " & IndicatorCount) 'should
increase by 1 from AddIndicator success
Debug.Print(" ")

Stop

IndicatorCount = swGtolFrame.GetIndicatorCount

```

```

iIndicator = 0
IFrame = 0
FrameCount = swGtol.GetFrameCount
Debug.Print("FrameCount is " & FrameCount & ": GetIndicator test")
For iFrame = 1 To FrameCount
    lBorderType = -1
    lTolType = -1
    sDatum = ""

    swGtolFrame1 = swGtol.GetFrame(iFrame)
    For iIndicator = 1 To IndicatorCount
        boolstatus = swGtolFrame1.GetIndicator(iIndicator, lBorderType, lTolType, sDatum)
        Debug.Print("Frame: " & iFrame & " Indicator index: " & iIndicator & " (BorderType as defined by
swGtolIndicatorBorderType_e = " & lBorderType & " " & "TolType as defined by swGtolGeomCharSymbol_e = " & lTolType & " " & "Datum
= " & sDatum & ")")
    Next
Next

Debug.Print(" ")

Stop

Debug.Print("Frame " & FrameCount & " DeleteIndicator test")
boolstatus = swGtolFrame.DeleteIndicator(IndicatorCount)
Debug.Print("DeleteIndicator return value is " & boolstatus)
IndicatorCount = swGtolFrame.GetIndicatorCount 'should decrease by 1, as indicator deleted
Debug.Print("IndicatorCount is " & IndicatorCount & " after DeleteIndicator")

Stop

Debug.Print(" ")
Debug.Print("Miscellaneous Indicator test:")
boolstatus = swGtolFrame.AddIndicator(swGtolIndicatorBorderType_e.swGtolIndicatorBorderType_OrientationPlane,
swGtolGeomCharSymbol_e.swGcsANG, sDatum) 'add indicator for different BorderType and TolType
Debug.Print("Supported swGtolIndicatorBorderType_OrientationPlane BorderType AddIndicator return value is " & boolstatus)
IndicatorCount = swGtolFrame.GetIndicatorCount 'should increase by 1
Debug.Print("Frame " & FrameCount & " IndicatorCount after one successful AddIndicator is " & IndicatorCount) 'should
increase by 1
boolstatus = swGtolFrame.GetIndicator(IndicatorCount, lBorderType, lTolType, sDatum)
Debug.Print("Frame " & FrameCount & " just added GetIndicator: " & " (BorderType = " & lBorderType & " " & "TolType = " &
lTolType & " " & "Datum = " & sDatum & ")")

If Not sDatum = "B" Then
    sDatum = "B"
Else
    sDatum = "A"
End If

boolstatus = swGtolFrame.SetIndicator(IndicatorCount,
swGtolIndicatorBorderType_e.swGtolIndicatorBorderType_IntersectionPlane, swGtolGeomCharSymbol_e.swGcsPERP, sDatum)
Debug.Print("Indicator " & IndicatorCount & " SetIndicator return value is " & boolstatus)
boolstatus = swGtolFrame.GetIndicator(IndicatorCount, lBorderType, lTolType, sDatum)
Debug.Print("Indicator " & IndicatorCount & " SetIndicator values " & " (BorderType = " & lBorderType & " " & "TolType = "
& lTolType & " " & "Datum = " & sDatum & ")")
Debug.Print(" ")

Stop

swGtolFrame = swGtol.GetFrame(swGtol.GetFrameCount)
Debug.Print("Frame " & 2 & " ToleranceType test")

boolstatus = swGtolFrame.GetFrameToleranceType(swToleranceType)
Debug.Print("GetFrameToleranceType " & "returned " & boolstatus)
Debug.Print("GetFrameToleranceType " & "before set returned tolerance type as defined by swGtolGeomCharSymbol_e: " &
swToleranceType)
boolstatus = swGtolFrame.SetFrameToleranceType(swGtolGeomCharSymbol_e.swGcsROUND)
Debug.Print("SetFrameToleranceType " & "returned " & boolstatus)

```

```

        boolstatus = swGtolFrame.GetFrameToleranceType(swTolType)
        Debug.Print("GetFrameToleranceType " & "returned " & boolstatus)
        Debug.Print("GetFrameToleranceType " & "after set returned tolerance type as defined by swGtolGeomCharSymbol_e: " &
swTolType)
        Debug.Print("")

        Stop

        sXMLString = swGtolFrame.GetSymbolXml
        Debug.Print("Symbol XML string: " & sXMLString)

        'Set a new symbol XML string of last added frame
        pos = -1
        pos = InStr(1, sXMLString, ".01") 'Note: converted Gtol has tolerance format of .01, not 0.01
        If Not pos = -1 Then
            sXMLString = Replace(sXMLString, ".01", ".02")
        Else
            pos = InStr(1, sXMLString, ".02")
            If Not pos = -1 Then
                sXMLString = Replace(sXMLString, ".02", ".01")
            End If
        End If

        boolstatus = swGtolFrame.SetSymbolXml(sXMLString)
        Debug.Print("New symbol XML string: " & sXMLString)

    End Sub
    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
End Class

```

Get GTol Witness Line Example (C#)

This example shows how to get the witness line of a geometric tolerance frame.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\GTolWitnessLine.slddrw
// 2. Rename the namespace to match the name of your C# project.
// 3. Select the geometric tolerance frame.
// 4. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window for the
// witness line start and end point coordinates.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GTolWitnessLine_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        SelectionMgr swSelMgr;
        GTol selGtol;
        int i;

        object coords;
        Double[] dbls;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            selGtol = (GTol)swSelMgr.GetSelectedObject6(1, 0);

            selGtol.SetPosition(0.433, 0.432, 0.0);
            swModel.ForceRebuild3(false);

            coords = selGtol.GetWitnessLine();
            dbls = (Double[])coords;

            for (i = 0; i < dbls.Length; i++)
            {
                Debug.Print(dbls[i].ToString());
            }
        }

        public SldWorks swApp;
    }
}
```

Get GTol Witness Line Example (VBA)

This example shows how to get the witness line of a geometric tolerance frame.

```
'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\GTolWitnessLine.slddrw
' 2. Select the frame.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. The position of the Gtol is modified.
' 2. Inspect the Immediate window for the witness line
'    start and end point coordinates.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim selGtol As SldWorks.Gtol
Dim i As Long
Dim params As Variant

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    Set selGtol = swSelMgr.GetSelectedObject6(1, 0)

    selGtol.SetPosition 0.433, 0.432, 0#
    swModel.ForceRebuild3 False

    params = selGtol.GetWitnessLine

    For i = 0 To UBound(params)
        Debug.Print params(i)
    Next i

End Sub
```

Get GTol Witness Line Example (VB.NET)

This example shows how to get the witness line of a geometric tolerance frame.

```
'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\GTolWitnessLine.slddrw
' 2. Select the geometric tolerance frame.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. The position of the Gtol is modified.
' 2. Inspect the Immediate window for the witness line
'    start and end point coordinates.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
```

```
    Dim swModelDocExt As ModelDocExtension
```

```
    Dim swSelMgr As SelectionMgr
```

```
    Dim selGtol As Gtol
```

```
    Dim i As Long
```

```
    Dim params As Object
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc
```

```
        swModelDocExt = swModel.Extension
```

```
        swSelMgr = swModel.SelectionManager
```

```
        selGtol = swSelMgr.GetSelectedObject6(1, 0)
```

```
        selGtol.SetPosition(0.433, 0.432, 0#)
```

```
        swModel.ForceRebuild3(False)
```

```
        params = selGtol.GetWitnessLine
```

```
        For i = 0 To UBound(params)
```

```
            Debug.Print(params(i))
```

```
        Next i
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Guide Curves in Loft Feature Example (C#)

This example shows how to get the guide curves in a loft feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part document
//    template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a new part document.
// 2. Creates a loft feature.
// 3. Prints to the Immediate window
//    the feature type and feature name of the loft
//    feature.
// 4. Accesses the guide curves in the loft feature.
// 5. Prints to the Immediate window whether the
//    loft is a boss feature, the number guide
//    curves in the loft, and the feature types of
//    the guide curves.
// 6. Releases access to the loft feature.
// 7. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace LoftGuideCurveCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        FeatureManager swFeatureManager;
        RefPlane swRefPlane;
        SketchManager swSketchManager;
        SketchSegment swSketchSegment;
        Feature swFeature;
        LoftFeatureData swLoftFeatureData;
        object pointArray = null;
        double[] points = null;
        object[] guideCurves = null;
        object guideCurve;
        int nbrGuideCurves;
        int[] guideCurvesTypes = null;
        int guideCurveType = 0;
        int i = 0;
        bool status = false;

        public void Main()
        {
            //Open new part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Create reference plane
            status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swRefPlane = (RefPlane)swFeatureManager.InsertRefPlane(8, 0.0635, 0, 0, 0, 0);
            swModel.ClearSelection2(true);

            //Create circle for loft feature
            status = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, true, 0, null, 0);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(-0.0, 0.0, 0.0, 0.003857, -0.009744, 0.0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            //Create another circle for loft feature
            status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchManager.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(-0.0, 0.0, 0.0, 0.014007, -0.029232, 0.0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            //Create sketch for guide curve
            status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            points = new double[9];
            points[0] = 0;
            points[1] = 0.0324150959148675;
            points[2] = 0;
            points[3] = 0.02176137524458;
            points[4] = 0.0209549481725162;
            points[5] = 0;
            points[6] = 0.0635;
            points[7] = 0.0104797609372824;
            points[8] = 0;
            pointArray = points;
            swSketchManager.InsertSketch(true);
            swSketchSegment = swSketchManager.CreateSpline((pointArray));
            swSketchManager.InsertSketch(true);

            //Create loft feature with guide curve
            status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID("Sketch3", "SKETCH", 0, 0, 0, true, 0, null, 0);
        }
    }
}
```

```

swModel.ClearSelection2(true);
status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0.0635, 0, -0.0104797609372824, false, 1, null, 0);
status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, -0.0324150959148675, true, 1, null, 0);
status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, true, 4098, null, 0);
swFeature = (Feature)swFeatureManager.InsertProtrusionBlend2(false, true, false, 1, 0, 0, 1, 1, true, true, false, 0, 0, 0, true, true, true,
(int)swGuideCurveInfluence_e.swGuideCurveInfluenceNextGlobal);
Debug.Print("Feature:");
Debug.Print("  Type: " + swFeature.GetTypeName2());
Debug.Print("  Name: " + swFeature.Name);

//Change the orientation of the view
swModel.ShowNamedView2("Isometric", 7);

//Access loft feature data, get guide curves,
//get feature type of guide curves, and release
//access to loft feature
swLoftFeatureData = (LoftFeatureData)swFeature.GetDefinition();
Debug.Print("  Boss feature: " + swLoftFeatureData.IsBossFeature());
nbrGuideCurves = swLoftFeatureData.GetGuideCurvesCount();
Debug.Print("    Number of guide curves: " + nbrGuideCurves);
status = swLoftFeatureData.AccessSelections(swModel, null);
Debug.Print("    Guide curve: ");
guideCurves = (object[])swLoftFeatureData.GuideCurves;
guideCurvesTypes = (int[])swLoftFeatureData.GetGuideCurvesType();
for (i = 0; i <= (nbrGuideCurves - 1); i++)
{
    guideCurve = (object)guideCurves[i];
    guideCurveType = (int)guideCurvesTypes[i];
    Debug.Print("      Type of feature: " + guideCurveType);
}
swLoftFeatureData.ReleaseSelectionAccess();

}

/// <summary>
///   The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Guide Curves in Loft Feature Example (VBA)

This example shows how to get the guide curves in a loft feature.

```
'-----
' Preconditions:
' 1. Verify that the specified part document
'    template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a new part document.
' 2. Creates a loft feature.
' 3. Prints to the Immediate window
'    the feature type and feature name of the loft
'    feature.
' 4. Accesses the guide curves in the loft feature.
' 5. Prints to the Immediate window whether the
'    loft is a boss feature, the number guide
'    curves in the loft, and the feature types
'    of the guide curves.
' 6. Releases access to the loft feature.
' 7. Examines the Immediate window.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swRefPlane As SldWorks.RefPlane
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swLoftFeatureData As SldWorks.LoftFeatureData
Dim pointArray As Variant
Dim points() As Double
Dim guideCurves As Variant
Dim guideCurve As Object
Dim nbrGuideCurves As Long
Dim i As Long
Dim status As Boolean

Sub main()

    'Open new part document
    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension

    'Create reference plane
    status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swRefPlane = swFeatureManager.InsertRefPlane(8, 0.0635, 0, 0, 0, 0)
    swModel.ClearSelection2 True

    'Create circle for loft feature
    status = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    Set swSketchSegment = swSketchManager.CreateCircle(-0#, 0#, 0#, 0.003857, -0.009744, 0#)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    'Create another circle for loft feature
    status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchManager.InsertSketch True
    Set swSketchSegment = swSketchManager.CreateCircle(-0#, 0#, 0#, 0.014007, -0.029232, 0#)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    'Create sketch for guide curve
    status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    ReDim points(0 To 8) As Double
    points(0) = 0
    points(1) = 3.24150959148675E-02
    points(2) = 0
    points(3) = 0.02176137524458
    points(4) = 2.09549481725162E-02
    points(5) = 0
    points(6) = 0.0635
    points(7) = 1.04797609372824E-02
    points(8) = 0
    pointArray = points
    swSketchManager.InsertSketch True
    Set swSketchSegment = swSketchManager.CreateSpline((pointArray))
    swSketchManager.InsertSketch True

    'Create loft feature with guide curve
    status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("Sketch3", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0.0635, 0, -1.04797609372824E-02, False, 1, Nothing, 0)
```

```

status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, -3.24150959148675E-02, True, 1, Nothing, 0)
status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, True, 4098, Nothing, 0)
Set swFeature = swFeatureManager.InsertProtusionBlend2(False, True, False, 1, 0, 0, 1, 1, True, True, False, 0, 0, 0, True, True, True,
swGuideCurveInfluence_e.swGuideCurveInfluenceNextGlobal)
Debug.Print "Feature:"
Debug.Print "  Type: " & swFeature.GetType2
Debug.Print "  Name: " & swFeature.Name

'Change the orientation of the view
swModel.ShowNamedView2 "Isometric", 7

'Access loft feature data, get guide curves,
'get feature type of guide curves, and release
'access to loft feature
Set swLoftFeatureData = swFeature.GetDefinition
Debug.Print ("  Boss feature: " & swLoftFeatureData.IsBossFeature)
nbrGuideCurves = swLoftFeatureData.GetGuideCurvesCount
Debug.Print ("  Number of guide curves: " & nbrGuideCurves)
status = swLoftFeatureData.AccessSelections(swModel, Nothing)
Debug.Print ("    Guide curve: ")
guideCurves = swLoftFeatureData.guideCurves
For i = 0 To (nbrGuideCurves - 1)
    Set guideCurve = guideCurves(i)
    Debug.Print ("      Type of feature: " & swLoftFeatureData.GetGuideCurvesType(i))
Next i
swLoftFeatureData.ReleaseSelectionAccess

End Sub

```

Get Guide Curves in Loft Feature Example (VB.NET)

This example shows how to get the guide curves in a loft feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part document
'    template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a new part document.
' 2. Creates a loft feature.
' 3. Prints to the Immediate window
'    the feature type and feature name of the loft
'    feature.
' 4. Accesses the guide curves in the loft feature.
' 5. Prints to the Immediate window whether the
'    loft is a boss feature, the number guide
'    curves in the loft, and the feature types of the
'    guide curves.
' 6. Releases access to the loft feature.
' 7. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swFeatureManager As FeatureManager
    Dim swRefPlane As RefPlane
    Dim swSketchManager As SketchManager
    Dim swSketchSegment As SketchSegment
    Dim swFeature As Feature
    Dim swLoftFeatureData As LoftFeatureData
    Dim pointArray As Object
    Dim points() As Double
    Dim guideCurves As Object
    Dim guideCurve As Object
    Dim nbrGuideCurves As Integer
    Dim i As Integer
    Dim status As Boolean

    Public Sub Main()

        'Open new part document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension

        'Create reference plane
        status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        swFeatureManager = swModel.FeatureManager
        swRefPlane = swFeatureManager.InsertRefPlane(8, 0.0635, 0, 0, 0, 0)
        swModel.ClearSelection2(True)

        'Create circle for loft feature
        status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateCircle(-0.0#, 0.0#, 0.0#, 0.003857, -0.009744, 0.0#)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

        'Create another circle for loft feature
        status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateCircle(-0.0#, 0.0#, 0.0#, 0.014007, -0.029232, 0.0#)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

        'Create sketch for guide curve
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        ReDim points(8)
        points(0) = 0
        points(1) = 0.0324150959148675
        points(2) = 0
        points(3) = 0.02176137524458
        points(4) = 0.0209549481725162
        points(5) = 0
        points(6) = 0.0635
        points(7) = 0.0104797609372824
        points(8) = 0
        pointArray = points
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateSpline((pointArray))
        swSketchManager.InsertSketch(True)

        'Create loft feature with guide curve
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0.0635, 0, -0.0104797609372824, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, -0.0324150959148675, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, True, 4098, Nothing, 0)
        swFeature = swFeatureManager.InsertProtrusionBlend2(False, True, False, 1, 0, 0, 1, 1, True, True, False, 0, 0, 0, True, True, True, swGuideCurveI

```

```

nfluence_e.swGuideCurveInfluenceNextGlobal)
Debug.Print("Feature:")
Debug.Print("  Type: " & swFeature.GetTypeName2)
Debug.Print("  Name: " & swFeature.Name)

'Change the orientation of the view
swModel.ShowNamedView2("Isometric", 7)

'Access loft feature data, get guide curves,
'get feature type of guide curves, and release
'access to loft feature
swLoftFeatureData = swFeature.GetDefinition
Debug.Print("  Boss feature: " & swLoftFeatureData.IsBossFeature)
nbrGuideCurves = swLoftFeatureData.GetGuideCurvesCount
Debug.Print("  Number of guide curves: " & nbrGuideCurves)
status = swLoftFeatureData.AccessSelections(swModel, Nothing)
Debug.Print("  Guide curve: ")
guideCurves = swLoftFeatureData.guideCurves
For i = 0 To (nbrGuideCurves - 1)
    guideCurve = guideCurves(i)
    Debug.Print("    Type of feature: " & swLoftFeatureData.GetGuideCurvesType(i))
Next i
swLoftFeatureData.ReleaseSelectionAccess()

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Guide Curves in Sweep Feature Example (C#)

This example shows how to get the guide curves in a sweep feature.

```
//-----
// Preconditions:
// 1. Verify that the part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a new part document.
// 2. Creates a sweep feature.
// 3. Gets the number of guide curves in the sweep
//    feature.
// 4. Accesses the guide curves in the sweep feature.
// 5. Gets the feature types of the guide curves.
// 6. Releases access of the sweep feature.
// 7. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GuideCurvesSweepFeatureCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureMgr = default(FeatureManager);
            SweepFeatureData swSweepFeatureData = default(SweepFeatureData);
            object pointArray = null;
            double[] points = null;
            object[] guideCurves = null;
            object guideCurve = null;
            int nbrGuideCurves = 0;
            int[] guideCurvesTypes = null;
            int guideCurveType = 0;
            bool status = false;
            long i = 0;

            //Create new model document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2017\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Sketch an ellipse for sweep profile
            swModel.ClearSelection2(true);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchMgr.InsertSketch(true);
            swSketchSegment = swSketchMgr.CreateEllipse(0, 0, 0, -0.064925207354862, 0, 0, 0, -0.0360377802938881, 0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);

            //Sketch a line for sweep path
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchMgr.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(0.0, 0.0, 0.0, 0.0, 0.059816, 0.0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);

            //Sketch a spline for sweep guide curve
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchMgr.InsertSketch(true);
            points = new double[6];
            points[0] = -0.064925207354862;
            points[1] = 0;
            points[2] = 0;
            points[3] = -0.00576005360247873;
            points[4] = 0.0595205538922803;
            points[5] = 0;
            pointArray = points;
            swSketchSegment = swSketchMgr.CreateSpline((pointArray));
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Unknown", "MANIPULATOR", -0.0481685228359519, 0.0168573405240843, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);
            swModel.ViewZoomtofit2();

            //Select the profile, path, and guide curve
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, true, 4, null, 0);
            status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, true, 2, null, 0);

            //Create the sweep feature
            swFeatureMgr = swModel.FeatureManager;
            swFeature = (Feature)swFeatureMgr.InsertProtrusionSwept4(false, false, (int)swTwistControlType_e.swTwistControlFollowPath, false, false, (int)
            swTangencyType_e.swTangencyNone, (int)swTangencyType_e.swTangencyNone, false, 0, 0, (int)swThinWallType_e.swThinWallOneDirection, 0, true, true, true, 0,
            true, false, 0, 0);
            Debug.Print("Feature type: " + swFeature.GetTypeName2());

            //Change the orientation of the view
            swModel.ShowNamedView2("Isometric", 7);

            //Access sweep feature data, get guide curves,
            //get feature type of guide curves, and release
            //access to sweep feature
            swSweepFeatureData = (SweepFeatureData)swFeature.GetDefinition();
```

```

        nbrGuideCurves = swSweepFeatureData.GetGuideCurvesCount();
        Debug.Print("  Number of guide curves: " + nbrGuideCurves);
        status = swSweepFeatureData.AccessSelections(swModel, null);
        Debug.Print("    Guide curve: ");
        guideCurves = (object[])swSweepFeatureData.GuideCurves;
        guideCurvesTypes = (int[])swSweepFeatureData.GetGuideCurvesType();
        for (i = 0; i <= (nbrGuideCurves - 1); i++)
        {
            guideCurve = (object)guideCurves[i];
            guideCurveType = (int)guideCurvesTypes[i];
            Debug.Print("      Type of feature as defined in swSelectType_e: " + guideCurveType);
        }
        swSweepFeatureData.ReleaseSelectionAccess();
    }

    /// <summary>
    ///   The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Get Guide Curves in Sweep Feature Example (VBA)

This example shows how to get the guide curves in a sweep feature.

```

'-----
' Preconditions:
' 1. Verify that the part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a new part document.
' 2. Creates a sweep feature.
' 3. Gets the number of guide curves in the sweep
'   feature.
' 4. Accesses the guide curves in the sweep feature.
' 5. Gets the feature types of the guide curves.
' 6. Releases access of the sweep feature.
' 7. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swSweepFeatureData As SldWorks.SweepFeatureData
Dim pointArray As Variant
Dim points() As Double
Dim guideCurves As Variant
Dim guideCurve As Object
Dim nbrGuideCurves As Long
Dim status As Boolean
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Create new model document
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2017\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension

    'Sketch an ellipse for sweep profile
    swModel.ClearSelection2 True
    Set swSketchMgr = swModel.SketchManager
    status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchMgr.InsertSketch True
    Set swSketchSegment = swSketchMgr.CreateEllipse(0, 0, 0, -0.064925207354862, 0, 0, 0, -3.60377802938881E-02, 0)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

    'Sketch a line for sweep path
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchMgr.InsertSketch True
    Set swSketchSegment = swSketchMgr.CreateLine(0#, 0#, 0#, 0#, 0.059816, 0#)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

    'Sketch a spline for sweep guide curve
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchMgr.InsertSketch True

    ReDim points(0 To 5) As Double
    points(0) = -0.064925207354862
    points(1) = 0
    points(2) = 0
    points(3) = -5.76005360247873E-03
    points(4) = 5.95205538922803E-02
    points(5) = 0
    pointArray = points
    Set swSketchSegment = swSketchMgr.CreateSpline((pointArray))
    swModel.ClearSelection2 True

    status = swModelDocExt.SelectByID2("Unknown", "MANIPULATOR", -4.81685228359519E-02, 1.68573405240843E-02, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True
    swModel.ViewZoomtofit2

    'Select the profile, path, and guide curve
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, True, 2, Nothing, 0)

    'Create the sweep feature
    Set swFeatureMgr = swModel.FeatureManager
    Set swFeature = swFeatureMgr.InsertProtrusionSwept4(False, False, swTwistControlType_e.swTwistControlFollowPath, False, False,
    swTangencyType_e.swTangencyNone, swTangencyType_e.swTangencyNone, False, 0, 0, swThinWallType_e.swThinWallOneDirection, 0, True, True, True, 0, True,
    False, 0, 0)
    Debug.Print "Feature type: " & swFeature.GetTypeName2

```

```

'Change the orientation of the view
swModel.ShowNamedView2 "**Isometric", 7

'Access sweep feature data, get guide curves,
'get feature type of guide curves, and release
'access to sweep feature
Set swSweepFeatureData = swFeature.GetDefinition
nbrGuideCurves = swSweepFeatureData.GetGuideCurvesCount
Debug.Print (" Number of guide curves: " & nbrGuideCurves)
status = swSweepFeatureData.AccessSelections(swModel, Nothing)
Debug.Print (" Guide curve: ")
guideCurves = swSweepFeatureData.guideCurves
For i = 0 To (nbrGuideCurves - 1)
    Set guideCurve = guideCurves(i)
    Debug.Print (" Type of feature as defined in swSelectType_e: " & swSweepFeatureData.GetGuideCurvesType(i))
Next i
swSweepFeatureData.ReleaseSelectionAccess

End Sub

```

Get Guide Curves in Sweep Feature Example (VB.NET)

This example shows how to get the guide curves in a sweep feature.

```

'-----
' Preconditions:
' 1. Verify that the part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a new part document.
' 2. Creates a sweep feature.
' 3. Gets the number of guide curves in the sweep
'    feature.
' 4. Accesses the guide curves in the sweep feature.
' 5. Gets the feature types of the guide curves.
' 6. Releases access of the sweep feature.
' 7. Examine the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeature As Feature
        Dim swFeatureMgr As FeatureManager
        Dim swSweepFeatureData As SweepFeatureData
        Dim pointArray As Object
        Dim points() As Double
        Dim guideCurves As Object
        Dim guideCurve As Object
        Dim nbrGuideCurves As Integer
        Dim status As Boolean
        Dim i As Integer

        'Create new model document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2017\templates\Part.prt", 0, 0, 0)
        swModelDocExt = swModel.Extension

        'Sketch an ellipse for sweep profile
        swModel.ClearSelection2(True)
        swSketchMgr = swModel.SketchManager
        status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchMgr.InsertSketch(True)
        swSketchSegment = swSketchMgr.CreateEllipse(0, 0, 0, -0.064925207354862, 0, 0, 0, -0.0360377802938881, 0)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

        'Sketch a line for sweep path
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchMgr.InsertSketch(True)
        swSketchSegment = swSketchMgr.CreateLine(0.0#, 0.0#, 0.0#, 0.0#, 0.059816, 0.0#)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

        'Sketch a spline for sweep guide curve
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchMgr.InsertSketch(True)
        ReDim points(5)
        points(0) = -0.064925207354862
        points(1) = 0
        points(2) = 0
        points(3) = -0.00576005360247873
        points(4) = 0.0595205538922803
        points(5) = 0
        pointArray = points
        swSketchSegment = swSketchMgr.CreateSpline((pointArray))
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Unknown", "MANIPULATOR", -0.0481685228359519, 0.0168573405240843, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)
        swModel.ViewZoomtofit2()

        'Select the profile, path, and guide curve
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, True, 4, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, True, 2, Nothing, 0)

        'Create the sweep feature
        swFeatureMgr = swModel.FeatureManager
        swFeature = swFeatureMgr.InsertProtrusionSwept4(False, False, swTwistControlType_e.swTwistControlFollowPath, False, False, swTangencyType_e.swTangencyNone, swTangencyType_e.swTangencyNone, False, 0, 0, swThinWallType_e.swThinWallOneDirection, 0, True, True, True, 0, True, False, 0, 0)
        Debug.Print("Feature type: " & swFeature.GetTypeName2)

        'Change the orientation of the view
        swModel.ShowNamedView2("Isometric", 7)

        'Access sweep feature data, get guide curves,
        'get feature type of guide curves, and release
        'access to sweep feature
        swSweepFeatureData = swFeature.GetDefinition
        nbrGuideCurves = swSweepFeatureData.GetGuideCurvesCount
        Debug.Print(" Number of guide curves: " & nbrGuideCurves)
    End Sub
End Class

```

```
status = swSweepFeatureData.AccessSelections(swModel, Nothing)
Debug.Print("    Guide curve: ")
guideCurves = swSweepFeatureData.GuideCurves
For i = 0 To (nbrGuideCurves - 1)
    guideCurve = guideCurves(i)
    Debug.Print("        Type of feature as defined in swSelectType_e: " & swSweepFeatureData.GetGuideCurvesType(i))
Next i
swSweepFeatureData.ReleaseSelectionAccess()

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Get Gusset Feature Data Example (C#)

This example shows how to get some data for a gusset feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Selects and gets the Gusset1 feature.
// 3. Gets some Gusset1 feature data.
// 4. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GussetFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            GussetFeatureData swGussetFeatureData = default(GussetFeatureData);
            bool status = false;
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            Face2 oneFace = null;
            Face2 twoFace = null;

            //Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\weldment_box3.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Select and get Gusset1
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Gusset1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swGussetFeatureData = (GussetFeatureData)swFeature.GetDefinition();

            //Access Gusset1 and get some feature data
            swGussetFeatureData.AccessSelections(swModel, null);
            Debug.Print("Profile type: " + swGussetFeatureData.ProfileType);
            Debug.Print("Thickness type: " + swGussetFeatureData.ThicknessType);
            Debug.Print("Thickness: " + swGussetFeatureData.Thickness);
            if (swGussetFeatureData.OffsetUsed == true)
            {
                Debug.Print("Profile offset distance: " + swGussetFeatureData.ProfileOffsetDistance);
            }
            else
            {
                Debug.Print("No profile offset used.");
            }
            status = swGussetFeatureData.GetSupportingFaces(out oneFace, out twoFace);
            status = oneFace.IsSame(twoFace);
            Debug.Print("Is the first supporting face the same as the second supporting face: " + status);
            swGussetFeatureData.ReleaseSelectionAccess();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Gusset Feature Data Example (VBA)

This example shows how to get some data for a gusset feature.

```
'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects and gets the Gusset1 feature.
' 3. Gets to some Gusset1 feature data.
' 4. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swGussetFeatureData As SldWorks.GussetFeatureData
Dim status As Boolean
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim oneFace As SldWorks.face2
Dim twoFace As SldWorks.face2

Sub main()

    Set swApp = Application.SldWorks

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\weldment_box3.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select and get Gusset1
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Gusset1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swGussetFeatureData = swFeature.GetDefinition

    'Access Gusset1 and get some feature data
    swGussetFeatureData.AccessSelections swModel, Nothing
    Debug.Print "Profile type: " & swGussetFeatureData.ProfileType
    Debug.Print "Thickness type: " & swGussetFeatureData.ThicknessType
    Debug.Print "Thickness: " & swGussetFeatureData.Thickness
    If swGussetFeatureData.OffsetUsed = True Then
        Debug.Print "Profile offset distance: " & swGussetFeatureData.ProfileOffsetDistance
    Else
        Debug.Print "No profile offset used."
    End If
    status = swGussetFeatureData.GetSupportingFaces(oneFace, twoFace)
    status = oneFace.IsSame(twoFace)
    Debug.Print "Is the first supporting face the same as the second supporting: " & status
    swGussetFeatureData.ReleaseSelectionAccess

End Sub
```

Get Gusset Feature Data Example (VB.NET)

This example shows how to get some data for a gusset feature.

```

-----
Preconditions:
1. Verify that the specified part document to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified part document.
2. Selects and gets the Gusset1 feature.
3. Gets some Gusset1 feature data.
4. Examine the Immediate window.
NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swGussetFeatureData As GussetFeatureData
        Dim status As Boolean
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim oneFace As Face2 = Nothing
        Dim twoFace As Face2 = Nothing

        'Open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\weldment_box3.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Select and get Gusset1
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Gusset1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swGussetFeatureData = swFeature.GetDefinition

        'Access Gusset1 and get some feature data
        swGussetFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Profile type: " & swGussetFeatureData.ProfileType)
        Debug.Print("Thickness type: " & swGussetFeatureData.ThicknessType)
        Debug.Print("Thickness: " & swGussetFeatureData.Thickness)
        If swGussetFeatureData.OffsetUsed = True Then
            Debug.Print("Profile offset distance: " & swGussetFeatureData.ProfileOffsetDistance)
        Else
            Debug.Print("No profile offset used.")
        End If
        status = swGussetFeatureData.GetSupportingFaces(oneFace, twoFace)
        status = oneFace.IsSame(twoFace)
        Debug.Print("Is the first supporting face the same as the second supporting face: " & status)
        swGussetFeatureData.ReleaseSelectionAccess()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Get Hatching Data Example (VBA)

This example shows how to get hatching data from a drawing view.

In a drawing view, hatching can be applied by the user to any face or, as in the case of a section view, automatically applied by the SOLIDWORKS Design software. Hatching applied by the user is accessed through the ISketchHatch object. In contrast, hatching in a section view is accessed through the IFaceHatch object.

```

'-----
' Preconditions:
' 1. Open public_documents\introsw\bolt-assembly.slddrw.
' 2. Select Section View A-A in the FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets hatching data.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swView As SldWorks.View
    Dim swSketch As SldWorks.Sketch
    Dim vFaceHatch As Variant
    Dim swFaceHatch As SldWorks.FaceHatch
    Dim swFace As SldWorks.Face2
    Dim vSketchHatch As Variant
    Dim swSketchHatch As SldWorks.SketchHatch
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc

    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    Set swSketch = swView.GetSketch
    Debug.Print "View = " & swView.Name
    Debug.Print "Type = " & swView.Type
    Debug.Print ""
    vFaceHatch = swView.GetFaceHatches
    If Not IsEmpty(vFaceHatch) Then
        Debug.Print "Face hatches:"
        Debug.Print ""
        For i = 0 To UBound(vFaceHatch)
            Set swFaceHatch = vFaceHatch(i)
            Set swFace = swFaceHatch.Face
            ' Cannot select a face because a face is in model
            bRet = swFace.Select2(True, 0)
            ' Get sketch hatch data
            ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
            Debug.Print "Angle = " & swFaceHatch.Angle * 57.3 & " degrees"
            Debug.Print "Color = " & swFaceHatch.Color
            Debug.Print "Definition = " & swFaceHatch.Definition
            Debug.Print "Layer = " & swFaceHatch.Layer
            Debug.Print "Pattern = " & swFaceHatch.Pattern
            Debug.Print "Scale = " & swFaceHatch.Scale2
            Debug.Print "SolidFill = " & swFaceHatch.SolidFill
            Debug.Print "-----"
        Next i
    End If

    vSketchHatch = swSketch.GetSketchHatches
    If Not IsEmpty(vSketchHatch) Then
        Debug.Print "Sketch hatches:"
        Debug.Print ""
        For i = 0 To UBound(vSketchHatch)
            Set swSketchHatch = vSketchHatch(i)
            Set swFace = swSketchHatch.GetFace
            ' Cannot select a face because a face is in model
            bRet = swFace.Select2(True, 0)
            ' Get sketch hatch data
            Debug.Print "Angle = " & swSketchHatch.Angle * 57.3 & " degrees"
            Debug.Print "Color = " & swSketchHatch.Color
            Debug.Print "ID = [" & swSketchHatch.GetID(0) & ", " & swSketchHatch.GetID(1) & "]"
            Debug.Print "Layer = " & swSketchHatch.Layer
            Debug.Print "LayerOverride = " & swSketchHatch.LayerOverride
            Debug.Print "Pattern = " & swSketchHatch.Pattern
            Debug.Print "Scale = " & swSketchHatch.Scale2
            Debug.Print "SolidFill = " & swSketchHatch.SolidFill
            Debug.Print "-----"
        Next i
    End If
End Sub

```

Get Heal Edges Feature Data Example (C#)

This example shows how to get some data of a heal edges feature.

```
//-----
// Preconditions:
// 1. Open a part document that contains a HealEdge1 feature.
// 2. Select the HealEdge1 feature.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets HealEdge1 feature data.
// 2. Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace HealEdgesFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            HealEdgesFeatureData swHealEdgesFeature = default(HealEdgesFeatureData);
            Feature swFeature = default(Feature);
            int before = 0;
            int after = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Get and print HealEdge1 feature data
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, 0);
            swHealEdgesFeature = (HealEdgesFeatureData)swFeature.GetDefinition();
            swModel.ClearSelection2(true);
            swHealEdgesFeature.AccessSelections(swModel, null);
            Debug.Print("Number of faces in this heal edges feature: " + swHealEdgesFeature.GetFacesCount());
            Debug.Print("Number of edges before and after healing: ");
            swHealEdgesFeature.GetEdgeInformation(out before, out after);
            Debug.Print(" Before: " + before);
            Debug.Print(" After : " + after);
            Debug.Print(" Maximum angle between the merged edges: " + swHealEdgesFeature.AngularTolerance);
            Debug.Print(" Maximum length of the merged edges: " + swHealEdgesFeature.EdgeLengthTolerance);
            swHealEdgesFeature.ReleaseSelectionAccess();

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Heal Edges Feature Data Example (VBA)

This example shows how to get some data of a heal edges feature.

```
'-----  
' Preconditions:  
' 1. Open a part document that contains a HealEdge1 feature.  
' 2. Select the HealEdge1 feature.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets HealEdge1 feature data.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swHealEdgesFeature As SldWorks.HealEdgesFeatureData  
Dim swFeature As SldWorks.Feature  
Dim before As Long  
Dim after As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    'Get and print HealEdge1 feature data  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeature = swSelMgr.GetSelectedObject6(1, 0)  
    Set swHealEdgesFeature = swFeature.GetDefinition  
    swModel.ClearSelection2 True  
    swHealEdgesFeature.AccessSelections swModel, Nothing  
    Debug.Print "Number of faces in this heal edges feature: " & swHealEdgesFeature.GetFacesCount  
    Debug.Print "Number of edges before and after healing: "  
    swHealEdgesFeature.GetEdgeInformation before, after  
    Debug.Print " Before: " & before  
    Debug.Print " After : " & after  
    Debug.Print " Maximum angle between the merged edges: " & swHealEdgesFeature.AngularTolerance  
    Debug.Print " Maximum length of the merged edges: " & swHealEdgesFeature.EdgeLengthTolerance  
    swHealEdgesFeature.ReleaseSelectionAccess  
  
End Sub
```

Get Heal Edges Feature Data Example (VB.NET)

This example shows how to get some data of a heal edges feature.

```

-----
Preconditions:
1. Open a part document that contains a HealEdge1 feature.
2. Select the HealEdge1 feature.
3. Open the Immediate window.

```

```

-----
Postconditions:
1. Gets HealEdge1 feature data.
2. Examine the Immediate window.
-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```

Partial Class SolidWorksMacro

```

```

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swHealEdgesFeature As HealEdgesFeatureData
        Dim swFeature As Feature
        Dim before As Integer
        Dim after As Integer

        swModel = swApp.ActiveDoc

        'Get and print HealEdge1 feature data
        swSelMgr = swModel.SelectionManager
        swFeature = swSelMgr.GetSelectedObject6(1, 0)
        swHealEdgesFeature = swFeature.GetDefinition
        swModel.ClearSelection2(True)
        swHealEdgesFeature.AccessSelections(swModel, Nothing)
        Debug.Print("Number of faces in this heal edges feature: " & swHealEdgesFeature.GetFacesCount)
        Debug.Print("Number of edges before and after healing: ")
        swHealEdgesFeature.GetEdgeInformation(before, after)
        Debug.Print(" Before: " & before)
        Debug.Print(" After : " & after)
        Debug.Print(" Maximum angle between the merged edges: " & swHealEdgesFeature.AngularTolerance)
        Debug.Print(" Maximum length of the merged edges: " & swHealEdgesFeature.EdgeLengthTolerance)
        swHealEdgesFeature.ReleaseSelectionAccess()
    End Sub

```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

```

End Class

```

Get Hidden Components Filenames Example (C#)

This example shows how to get the filenames of the hidden components in an assembly.

```
//-----
// Preconditions:
// 1. In SOLIDWORKS Design , click File > Open, and browse to
//    public_documents\samples\tutorial\routing-pipes.
// 2. Click ball valve with flanges.sldasm > Mode >
//    Large Design Review > Open > OK.
//
//    NOTE: If the path to the Design Library does not exist,
//    then multiple dialogs informing you that SOLIDWORKS Design is unable
//    to locate might be displayed some components. Click No or OK
//    to close these dialogs.
//
// 3. Click ball valve-1 in the FeatureManager design tree
//    and click Selective Open > Selective Open > Selected components >
//    Open Selected > OK.
//
//    NOTE: Only the selected components are loaded. Components
//    not selected are not loaded and not visible in the
//    SOLIDWORKS Design graphics area.
//
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Does not load the slip on weld flange components because
//    they are hidden.
// 2. Examine the graphics area and Immediate window.
//
// NOTE: Because this assembly elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Windows.Forms;
namespace Macro1.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {

            ModelDoc2 swModel;
            AssemblyDoc swAssembly;
            Boolean boolStatus;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)swModel;
            boolStatus = swAssembly.HasUnloadedComponents();
            if (boolStatus == true)
            {
                object oPaths = null;
                object oRefdConfigs = null;
                object oReasons = null;
                object oDocTypes = null;
                object oNames = null;

                oNames = swAssembly.GetUnloadedComponentNames(out oPaths, out oRefdConfigs, out oReasons, out oDocTypes);

                string[] Paths = null;
                string[] RefdConfigs = null;
                int[] Reasons = null;
                int[] DocTypes = null;
                string[] Names = null;

                Paths = (string[])oPaths;
                RefdConfigs = (string[]) oRefdConfigs;
```

```

Reasons = (int[]) oReasons;
DocTypes =(int[]) oDocTypes;
Names = (string[]) oNames;

if (!(Paths is System.Array) & (RefdConfigs is System.Array) & (Reasons is System.Array) & (DocTypes is
System.Array) & (Names is System.Array))
{
    MessageBox.Show("Error: Non-array parameter!");
    Debug.Assert(false);
    return;
}
if ((Paths.Length != RefdConfigs.Length) | (Paths.Length != Reasons.Length) | (Paths.Length != DocTypes.Length) |
(Paths.Length != Names.Length))
{
    MessageBox.Show("Error: Lengths of the arrays do not match!");
    Debug.Assert(false);
    return;
}

for (int index = 0; index < Paths.Length; index++)
{
    string debugMessage = null;
    debugMessage = index + ": ";
    swDocumentTypes_e eDocType;
    eDocType = (swDocumentTypes_e)(DocTypes[index]);
    switch (eDocType)
    {
        case swDocumentTypes_e.swDocNONE:
            debugMessage = debugMessage + "The document ";
            break;
        case swDocumentTypes_e.swDocPART:
            debugMessage = debugMessage + "The part ";
            break;
        case swDocumentTypes_e.swDocASSEMBLY:
            debugMessage = debugMessage + "The assembly ";
            break;
        case swDocumentTypes_e.swDocDRAWING:
            debugMessage = debugMessage + "The drawing ";
            break;
        case swDocumentTypes_e.swDocSDM:
            debugMessage = debugMessage + "The SDM ";
            break;
        default:
            debugMessage = debugMessage + "The document is an unknown type ";
            break;
    }
    debugMessage = debugMessage + Paths[index] + " was not loaded because it is ";
    bool bUnloadedBecauseHidden = false;
    if (Reasons[index] == 1)
        bUnloadedBecauseHidden = true;
    else
        bUnloadedBecauseHidden = false;

    if (bUnloadedBecauseHidden)
    {
        debugMessage = debugMessage + "hidden.";
    }
    else
    {
        debugMessage = debugMessage + "suppressed.";
    }
    Debug.Print(debugMessage);
}

```

```
    }

}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}
```

Get Hidden Components Filenames Example (VBA)

This example shows how to get the filenames of the components hidden in an assembly.

```
'-----
' Preconditions:
' 1. In SOLIDWORKS Design, click File > Open, and browse to
'    public_documents\samples\tutorial\routing-pipes.
' 2. Click ball valve with flanges.sldasm > Mode >
'    Large Design Review > Open > OK.
'
' NOTE: If the path to the Design Library does not exist,
' then multiple dialogs informing you that SOLIDWORKS Design is unable
' to locate might be displayed some components. Click No or OK
' to close these dialogs.
'
' 3. Click ball valve-1 in the FeatureManager design tree
' and click Selective Open > Selective Open > Selected components >
' Open Selected > OK.
'
' NOTE: Only the selected components are loaded. Components
' not selected are not loaded and not visible in the
' SOLIDWORKS Design graphics area.
'
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Does not load the slip on weld flange components because
' they are hidden.
' 2. Examine the graphics area and Immediate window.
'
' NOTE: Because this assembly elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks

Sub main()

    Set swApp = Application.SldWorks
    Dim swAssembly As SldWorks.AssemblyDoc
    Set swAssembly = swApp.ActiveDoc

    If swAssembly.HasUnloadedComponents Then
        Dim vPaths As Variant
        Dim vRefdConfigs As Variant
        Dim vReasons As Variant
        Dim vDocTypes As Variant
        Dim vNames As Variant

        vNames = swAssembly.GetUnloadedComponentNames(vPaths, vRefdConfigs, vReasons, vDocTypes)

        If IsEmpty(vPaths) Or IsEmpty(vRefdConfigs) Or IsEmpty(vReasons) Or IsEmpty(vDocTypes) Or IsEmpty(vNames) Then
            MsgBox "Error: Empty VARIANT parameter!"
            Debug.Assert False
            Exit Sub
        End If

        If Not (IsArray(vPaths) And IsArray(vRefdConfigs) And IsArray(vReasons) And IsArray(vDocTypes) And IsArray(vNames)) Then
            MsgBox "Error: Non-array VARIANT parameter!"
            Debug.Assert False
            Exit Sub
        End If

        If (LBound(vPaths) <> LBound(vRefdConfigs)) Or (LBound(vPaths) <> LBound(vReasons)) Or (LBound(vPaths) <> LBound(vDocTypes)) Or (LBound(vPaths) <> LBound(vNames)) Then
            MsgBox "Error: Array lower bounds do not match!"
            Debug.Assert False
            Exit Sub
        End If

        If (UBound(vPaths) <> UBound(vRefdConfigs)) Or (UBound(vPaths) <> UBound(vReasons)) Or (UBound(vPaths) <> UBound(vDocTypes)) Or (UBound(vPaths) <> UBound(vNames)) Then
            MsgBox "Error: Array upper bounds do not match!"
            Debug.Assert False
            Exit Sub
        End If

        Dim index As Integer
        For index = LBound(vNames) To UBound(vNames)
            Dim debugMessage As String
            debugMessage = index & ": "
            Dim eDocType As swDocumentTypes_e
            eDocType = vDocTypes(index)
            Select Case eDocType
            Case swDocNONE
                debugMessage = debugMessage & "The document "
            Case swDocPART
                debugMessage = debugMessage & "The part "
            Case swDocASSEMBLY
                debugMessage = debugMessage & "The assembly "
            Case swDocDRAWING
                debugMessage = debugMessage & "The drawing "
            Case swDocSDM
                debugMessage = debugMessage & "The SDM "
            Case Else
            End Select
        Next index
    End If
End Sub
```

```
        debugMessage = debugMessage & "The document of unknown type "
    End Select

    debugMessage = debugMessage & vPaths(index) & " was not loaded because it is "

    Dim bUnloadedBecauseHidden As Boolean
    bUnloadedBecauseHidden = vReasons(index)
    If bUnloadedBecauseHidden Then
        debugMessage = debugMessage & "hidden. "
    Else
        debugMessage = debugMessage & "suppressed."
    End If
    Debug.Print debugMessage
Next

End If

End Sub
```

Get Hidden Components Filenames Example (VB.NET)

This example shows how to get the filenames of the hidden components in an assembly.

```
'-----
' Preconditions:
' 1. In SOLIDWORKS Design, click File > Open, and browse to
'    public_documents\samples\tutorial\routing-pipes.
' 2. Click ball valve with flanges.sldasm > Mode >
'    Large Design Review > Open > OK.
'
' NOTE: If the path to the Design Library does not exist,
' then multiple dialogs informing you that SOLIDWORKS Design is unable
' to locate might be displayed some components. Click No or OK
' to close these dialogs.
'
' 3. Click ball valve-1 in the FeatureManager design tree
' and click Selective Open > Selective Open > Selected components >
' Open Selected > OK.
'
' NOTE: Only the selected components are loaded. Components
' not selected are not loaded and not visible in the
' SOLIDWORKS Design graphics area.
'
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Does not load the slip on weld flange components because
'    they are hidden.
' 2. Examine the graphics area and Immediate window.
'
' NOTE: Because this assembly elsewhere, do not save changes.
'-----
```

Option Explicit On

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System

Imports System.Diagnostics

Partial Class SolidWorksMacro

Public Sub main()

Dim swAssembly As AssemblyDoc

swAssembly = swApp.ActiveDoc

If swAssembly.HasUnloadedComponents Then

Dim oPaths As Object = Nothing

Dim oRefdConfigs As Object = Nothing

Dim oReasons As Object = Nothing

Dim oDocTypes As Object = Nothing

Dim oNames As Object = Nothing

oNames = swAssembly.GetUnloadedComponentNames(oPaths, oRefdConfigs, oReasons, oDocTypes)

Dim Paths() As String = Nothing

Dim RefdConfigs() As String = Nothing

Dim Reasons() As Integer = Nothing

Dim DocTypes() As Integer = Nothing

Dim Names() As String = Nothing

Paths = oPaths

RefdConfigs = oRefdConfigs

Reasons = oReasons

DocTypes = oDocTypes

Names = oNames

If Not (IsArray(Paths) And IsArray(RefdConfigs) And IsArray(Reasons) And IsArray(DocTypes) And IsArray(Names)) Then
MsgBox("Error: Non-array parameter!")

```

        Debug.Assert(False)
    Exit Sub
End If

    If (LBound(Paths) <> LBound(RefdConfigs)) Or (LBound(Paths) <> LBound(Reasons)) Or (LBound(Paths) <> LBound(DocTypes)
Or (LBound(Paths) <> LBound(Names))) Then
        MsgBox("Error: Array lower bounds do not match!")
        Debug.Assert(False)
    Exit Sub
End If

    If (UBound(Paths) <> UBound(RefdConfigs)) Or (UBound(Paths) <> UBound(Reasons)) Or (UBound(Paths) <> UBound(DocTypes)
Or (UBound(Paths) <> UBound(Names))) Then
        MsgBox("Error: Array upper bounds do not match!")
        Debug.Assert(False)
    Exit Sub
End If

Dim index As Integer
For index = LBound(Paths) To UBound(Paths)
    Dim debugMessage As String
    debugMessage = index & ": "

    Dim eDocType As swDocumentTypes_e
    eDocType = DocTypes(index)
    Select Case eDocType
        Case swDocumentTypes_e.swDocNONE
            debugMessage = debugMessage & "The document "
        Case swDocumentTypes_e.swDocPART
            debugMessage = debugMessage & "The part "
        Case swDocumentTypes_e.swDocASSEMBLY
            debugMessage = debugMessage & "The assembly "
        Case swDocumentTypes_e.swDocDRAWING
            debugMessage = debugMessage & "The drawing "
        Case swDocumentTypes_e.swDocSDM
            debugMessage = debugMessage & "The SDM "
        Case Else
            debugMessage = debugMessage & "The document is an unknown type "
    End Select

    debugMessage = debugMessage & Paths(index) & " was not loaded because it is "

    Dim bUnloadedBecauseHidden As Boolean
    bUnloadedBecauseHidden = Reasons(index)
    If bUnloadedBecauseHidden Then
        debugMessage = debugMessage & "hidden."
    Else
        debugMessage = debugMessage & "suppressed."
    End If

    Debug.Print(debugMessage)
Next
End If

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Get Hole Series Information Example (C#)

This example shows how to get information about a hole series.

```
//-----
// Preconditions:
// 1. Open an assembly that contains a hole series feature.
// 2. Configure holes in the SOLIDWORKS Toolbox.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace HoleSeriesFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swDoc;
        SelectionMgr SelMgr;
        bool boolstatus;
        Feature swFeat;
        HoleSeriesFeatureData2 swFeatData;
        Face2 swFace;

        Entity swEnt;

        public void Main()
        {
            swDoc = (ModelDoc2)swApp.ActiveDoc;
            SelMgr = (SelectionMgr)swDoc.SelectionManager;
            swFeat = (Feature)swDoc.FirstFeature();

            while ((swFeat != null))
            {
                Debug.Print(swFeat.GetTypeName() + ", " + swFeat.Name);
                if (swFeat.GetTypeName() == "HoleSeries")
                {
                    swFeatData = (HoleSeriesFeatureData2)swFeat.GetDefinition();
                    Debug.Print("  Standard:", swFeatData.Standard);

                    int i = 0;

                    for (i = 0; i <= swFeatData.GetComponentsCount() - 1; i++)
                    {
                        Debug.Print("    Component " + i);
                        Debug.Print("      Type:", swFeatData.get_Type(i));
                        Debug.Print("      Size:", swFeatData.get_Size(i));
                    }

                    Debug.Print("  FastenerMaterial:", swFeatData.Material);

                    // Bolt and nut information causes an error
                    // if SOLIDWORKS Toolbox is not configured
                    Debug.Print("  BoltHeadDiameter:", swFeatData.BoltHeadDiameter);
                    Debug.Print("  BoltDiameter:", swFeatData.BoltDiameter);
                    Debug.Print("  NutDiameter:", swFeatData.NutDiameter);
                    Debug.Print("  FastenerPreload:", swFeatData.Preload);
                    Debug.Print("  FastenerDefaultUnits:", swFeatData.FastenerDefaultUnits);
                    Debug.Print("  FastenerTopHoleType:", swFeatData.FastenerTopHoleType);
                    Debug.Print("  FastenerBottomHoleType:", swFeatData.FastenerBottomHoleType);
                    Debug.Print("  FastenerHoleCount:", swFeatData.FastenerHoleCount);

                    // End face
                    swFeatData.AccessSelections(swDoc, null);
                    swFace = (Face2)swFeatData.EndFace;
                    swEnt = (Entity)swFace;

                    if ((swEnt != null))
                    {
                        boolstatus = swEnt.Select4(false, null);
                        Debug.Print("  Face selection = " + boolstatus);
                    }

                    // Sketch points
                    int ncount = 0;
                    ncount = swFeatData.GetSketchPointsCount();
                    Debug.Print("  Sketch Point Count = " + ncount);

                    object[] vPtArr = null;
                    vPtArr = (object[])swFeatData.GetSketchPoints();
                }
            }
        }
    }
}
```

```

SketchPoint swSketchPoint = default(SketchPoint);

for (i = 0; i < ncount; i++)
{
    swSketchPoint = (SketchPoint)vPtArr[i];
    swSketchPoint.Select4(false, null);
}

// Components
ncount = swFeatData.GetComponentsCount();
Debug.Print(" ComponentsCount Count = " + ncount);
vPtArr = (object[])swFeatData.GetComponents();
Component swComp = null;

for (i = 0; i < ncount; i++)
{
    swComp = (Component)vPtArr[i];
    if ((swComp != null))
    {
        Debug.Print(" " + swComp.Name2 + " --> " + swComp.GetPathName());
    }
    else
    {
        Debug.Print(" Could not get component.");
    }
}

swFeatData.ReleaseSelectionAccess();
}

swFeat = (Feature)swFeat.GetNextFeature();
}

}

public SldWorks swApp;
}
}

```

Get Hole Series Information Example (VBA)

This example shows how to get information about a hole series.

```
'-----
' Preconditions:
' 1. Open an assembly that contains a hole series feature.
' 2. Configure holes and fasteners in the SOLIDWORKS Toolbox.
'
' Postconditions: Inspect the Immediate window.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swDoc As SldWorks.ModelDoc2
Dim SelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Dim swFeat As SldWorks.Feature
Dim swFeatData As SldWorks.HoleSeriesFeatureData2
Dim swFace As SldWorks.Face2
Dim swEnt As SldWorks.Entity
```

Sub main()

```
    Set swApp = Application.SldWorks
    Set swDoc = swApp.ActiveDoc
    Set SelMgr = swDoc.SelectionManager
    Set swFeat = swDoc.FirstFeature

    Do While Not swFeat Is Nothing
        Debug.Print swFeat.GetTypeName & ", " & swFeat.Name
        If swFeat.GetTypeName = "HoleSeries" Then
            Set swFeatData = swFeat.GetDefinition
            Debug.Print "Design standard:", swFeatData.Standard

            Dim i As Long
            For i = 0 To swFeatData.GetComponentsCount - 1
                Debug.Print "Component " & i
                Debug.Print "  Type:", swFeatData.Type(i)
                Debug.Print "  Size:", swFeatData.Size(i)
            Next

            Debug.Print "FastenerMaterial:", swFeatData.Material

            ' Bolt and nut information causes an error
            ' if SOLIDWORKS Toolbox is not configured
            Debug.Print "BoltHeadDiameter:", swFeatData.BoltHeadDiameter
            Debug.Print "BoltDiameter:", swFeatData.BoltDiameter
            Debug.Print "NutDiameter:", swFeatData.NutDiameter
            Debug.Print "FastenerPreload:", swFeatData.Preload
            Debug.Print "FastenerDefaultUnits:", swFeatData.FastenerDefaultUnits
            Debug.Print "FastenerTopHoleType:", swFeatData.FastenerTopHoleType
            Debug.Print "FastenerBottomHoleType:", swFeatData.FastenerBottomHoleType
            Debug.Print "FastenerHoleCount:", swFeatData.FastenerHoleCount

            ' End face
            swFeatData.AccessSelections swDoc, Nothing
            Set swFace = swFeatData.EndFace
            Set swEnt = swFace

            If Not swEnt Is Nothing Then
                boolstatus = swEnt.Select4(False, Nothing)
                Debug.Print " Face selection = " & boolstatus
            End If

            ' Sketch points
            Dim ncount As Long
            ncount = swFeatData.GetSketchPointsCount
            Debug.Print " Sketch Point Count = " & ncount

            Dim vPtArr As Variant
            vPtArr = swFeatData.GetSketchPoints

            Dim swSketchPoint As SldWorks.SketchPoint

            For i = 0 To i = ncount - 1
                Set swSketchPoint = vPtArr(i)
                swSketchPoint.Select4 False, Nothing
            Next

            ' Components
            ncount = swFeatData.GetComponentsCount
            Debug.Print "ComponentsCount Count = " & ncount
            vPtArr = swFeatData.GetComponents
            Dim swComp As Object

            For i = 0 To i = ncount - 1
                Set swComp = vPtArr(i)
                If Not swComp Is Nothing Then
                    Debug.Print " " & swComp.Name2 & " --> " & swComp.GetPathName
                Else
                    Debug.Print " Could not get component."
                End If
            Next
        End While
    End Sub
```

```
        swFeatData.ReleaseSelectionAccess  
    End If  
    Set swFeat = swFeat.GetNextFeature  
Loop  
  
End Sub
```

Get Hole Series Information Example (VB.NET)

This example shows how to get information about a hole series.

```

'-----
' Preconditions:
' 1. Open an assembly that contains a hole series feature.
' 2. Configure holes in the SOLIDWORKS Toolbox.
'-----
' Postconditions: Inspect the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swDoc As ModelDoc2
    Dim SelMgr As SelectionMgr
    Dim boolstatus As Boolean
    Dim swFeat As Feature
    Dim swFeatData As HoleSeriesFeatureData2
    Dim swFace As Face2
    Dim swEnt As Entity

    Sub main()

        swDoc = swApp.ActiveDoc
        SelMgr = swDoc.SelectionManager
        swFeat = swDoc.FirstFeature

        Do While Not swFeat Is Nothing
            Debug.Print(swFeat.GetTypeName & ", " & swFeat.Name)
            If swFeat.GetTypeName = "HoleSeries" Then
                swFeatData = swFeat.GetDefinition
                Debug.Print(" Standard:", swFeatData.Standard)

                Dim i As Integer

                For i = 0 To swFeatData.GetComponentsCount - 1
                    Debug.Print(" Component " & i)
                    Debug.Print(" Type:", swFeatData.Type(i))
                    Debug.Print(" Size:", swFeatData.Size(i))
                Next

                Debug.Print(" FastenerMaterial:", swFeatData.Material)

                ' Bolt and nut information causes an error
                ' if SOLIDWORKS Toolbox is not configured
                Debug.Print(" BoltHeadDiameter:", swFeatData.BoltHeadDiameter)
                Debug.Print(" BoltDiameter:", swFeatData.BoltDiameter)
                Debug.Print(" NutDiameter:", swFeatData.NutDiameter)
                Debug.Print(" FastenerPreload:", swFeatData.Preload)
                Debug.Print(" FastenerDefaultUnits:", swFeatData.FastenerDefaultUnits)
                Debug.Print(" FastenerTopHoleType:", swFeatData.FastenerTopHoleType)
                Debug.Print(" FastenerBottomHoleType:", swFeatData.FastenerBottomHoleType)
                Debug.Print(" FastenerHoleCount:", swFeatData.FastenerHoleCount)

                ' End face
                swFeatData.AccessSelections(swDoc, Nothing)
                swFace = swFeatData.EndFace
                swEnt = swFace

                If Not swEnt Is Nothing Then
                    boolstatus = swEnt.Select4(False, Nothing)
                    Debug.Print(" Face selection = " & boolstatus)
                End If

                ' Sketch points
                Dim ncount As Integer
                ncount = swFeatData.GetSketchPointsCount
                Debug.Print(" Sketch Point Count = " & ncount)

                Dim vPtArr As Object
                vPtArr = swFeatData.GetSketchPoints

                Dim swSketchPoint As SketchPoint

                For i = 0 To i = ncount - 1
                    swSketchPoint = vPtArr(i)
                    swSketchPoint.Select4(False, Nothing)
                Next

                ' Components
                ncount = swFeatData.GetComponentsCount
                Debug.Print(" ComponentsCount Count = " & ncount)
                vPtArr = swFeatData.GetComponents
            End While
        End Sub
    End Class

```

```
Dim swComp As Component

For i = 0 To i = ncount - 1
    swComp = vPtArr(i)
    If Not swComp Is Nothing Then
        Debug.Print(" " & swComp.Name2 & " --> " & swComp.GetPathName)
    Else
        Debug.Print(" Could not get component.")
    End If
Next

swFeatData.ReleaseSelectionAccess()

End If

swFeat = swFeat.GetNextFeature

Loop

End Sub

Public swApp As SldWorks

End Class
```

Get ID of Active Configuration or Current Drawing Sheet Example (C#)

This example shows how to get the name and ID of the active configuration of a part or assembly or the current sheet of a drawing.

NOTE: A unique ID is assigned to each configuration and drawing. This ID does not change when the name of the configuration or drawing sheet is changed.

```

//-----
// Preconditions:
// 1. Open a part, assembly, or drawing document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Changes either the active configuration's name or
// the current sheet's name to Test. However, the
// document's ID is unchanged.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Configuration swConfig = default(Configuration);
            DrawingDoc swDrawingDoc = default(DrawingDoc);
            Sheet swSheet = default(Sheet);
            int nDocType = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Get type of model document
            if (swModel.GetType() == 1)
            {
                nDocType = (int)swDocumentTypes_e.swDocPART;
            }
            else if (swModel.GetType() == 2)
            {
                nDocType = (int)swDocumentTypes_e.swDocASSEMBLY;
            }
            else if (swModel.GetType() == 3)
            {
                nDocType = (int)swDocumentTypes_e.swDocDRAWING;
            }
            else
            {
                // Not a SOLIDWORKS Design model document,
                // so exit macro
                return;
            }

            // If a part or assembly document,
            // then get the name of it and its ID
            if (nDocType != (int)swDocumentTypes_e.swDocDRAWING)
            {
                swConfig = (Configuration)swModel.GetActiveConfiguration();
                if ((swConfig != null))
                {
                    Debug.Print("Active configuration's name = " + swConfig.Name + ", ID = " + swConfig.GetID());
                    // Change the active configuration's name
                    swConfig.Name = "Test";
                    // Test to see if the ID remains the same
                    // after changing the name of the configuration
                    Debug.Print("Active configuration's new name = " + swConfig.Name + ", ID = " + swConfig.GetID());
                }
                return;
            }

            // A drawing sheet must be active,
            // so get its name and ID
            swDrawingDoc = (DrawingDoc)swModel;
            swSheet = (Sheet)swDrawingDoc.GetCurrentSheet();
            if ((swSheet != null))
            {
                Debug.Print("Current sheet's name = " + swSheet.GetName() + ", ID = " + swSheet.GetID());
                // Change current sheet's name
                swSheet.SetName("Test");
                // Test to see if the ID remains the same
                // after changing the name of the sheet
                Debug.Print("Current sheet's new name = " + swSheet.GetName() + ", ID = " + swSheet.GetID());
            }
        }

        public SldWorks swApp;
    }
}

```

Get ID of Active Configuration or Current Drawing Sheet Example (VBA)

This example shows how to get the name and ID of the active configuration of a part or assembly or the current sheet of a drawing.

NOTE: A unique ID is assigned to each configuration and drawing. This ID does not change when the name of the configuration or drawing sheet is changed.

```

'-----
' Preconditions:
' 1. Open a part, assembly, or drawing document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Changes either the active configuration's name or
'    the current sheet's name to Test. However, the
'    document's ID is unchanged.
' 2. Examine the Immediate window.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swConfig As SldWorks.Configuration
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swSheet As SldWorks.Sheet
Dim nDocType As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Get type of model document
    If swModel.GetType = 1 Then
        nDocType = swDocPART
    ElseIf swModel.GetType = 2 Then
        nDocType = swDocASSEMBLY
    ElseIf swModel.GetType = 3 Then
        nDocType = swDocDRAWING
    Else
        ' Not a SOLIDWORKS Design model document,
        ' so exit macro
        Exit Sub
    End If

    ' If a part or assembly document,
    ' then get the name of it and its ID
    If nDocType <> swDocDRAWING Then
        Set swConfig = swModel.GetActiveConfiguration
        If Not swConfig Is Nothing Then
            Debug.Print "Active configuration's name = " & swConfig.Name & ", ID = " & swConfig.GetID
            ' Change the active configuration's name
            swConfig.Name = "Test"
            ' Test to see if the ID remains the same
            ' after changing the name of the configuration
            Debug.Print "Active configuration's new name = " & swConfig.Name & ", ID = " & swConfig.GetID
        End If
        Exit Sub
    End If

    ' A drawing sheet must be active,
    ' so get its name and ID
    Set swDrawingDoc = swModel
    Set swSheet = swDrawingDoc.GetCurrentSheet
    If Not swSheet Is Nothing Then
        Debug.Print "Current sheet's name = " & swSheet.GetName & ", ID = " & swSheet.GetID
        ' Change current sheet's name
        swSheet.SetName "Test"
        ' Test to see if the ID remains the same
        ' after changing the name of the sheet
        Debug.Print "Current sheet's new name = " & swSheet.GetName & ", ID = " & swSheet.GetID
    End If
End Sub

```

Get ID of Active Configuration or Current Drawing Sheet Example (VB.NET)

This example shows how to get the name and ID of the active configuration of a part or assembly or the current sheet of a drawing.

NOTE: A unique ID is assigned to each configuration and drawing. This ID does not change when the name of the configuration or drawing sheet is changed.

```

'-----
' Preconditions:
' 1. Open a part, assembly, or drawing document.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Changes either the active configuration's name or
'    the current sheet's name to Test. However, the
'    document's ID is unchanged.
' 2. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swConfig As Configuration
        Dim swDrawingDoc As DrawingDoc
        Dim swSheet As Sheet
        Dim nDocType As Integer

        swModel = SWApp.ActiveDoc

        ' Get type of model document
        If swModel.GetType = 1 Then
            nDocType = swDocumentTypes_e.swDocPART
        ElseIf swModel.GetType = 2 Then
            nDocType = swDocumentTypes_e.swDocASSEMBLY
        ElseIf swModel.GetType = 3 Then
            nDocType = swDocumentTypes_e.swDocDRAWING
        Else
            ' Not a SOLIDWORKS Design model document,
            ' so exit macro
            Exit Sub
        End If

        ' If a part or assembly document,
        ' then get the name of it and its ID
        If nDocType <> swDocumentTypes_e.swDocDRAWING Then
            swConfig = swModel.GetActiveConfiguration
            If Not swConfig Is Nothing Then
                Debug.Print("Active configuration's name = " & swConfig.Name & ", ID = " & swConfig.GetID)
                ' Change the active configuration's name
                swConfig.Name = "Test"
                ' Test to see if the ID remains the same
                ' after changing the name of the configuration
                Debug.Print("Active configuration's new name = " & swConfig.Name & ", ID = " & swConfig.GetID)
            End If
            Exit Sub
        End If

        ' A drawing sheet must be active,
        ' so get its name and ID
        swDrawingDoc = swModel
        swSheet = swDrawingDoc.GetCurrentSheet
        If Not swSheet Is Nothing Then
            Debug.Print("Current sheet's name = " & swSheet.GetName & ", ID = " & swSheet.GetID)
            ' Change current sheet's name
            swSheet.SetName("Test")
            ' Test to see if the ID remains the same
            ' after changing the name of the sheet
            Debug.Print("Current sheet's new name = " & swSheet.GetName & ", ID = " & swSheet.GetID)
        End If

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

Public swApp As SldWorks

End Class

```

Get Imported Curve Feature Data Example (C#)

This example shows how to get the number of curves in an imported curve feature.

```
//-----
// Preconditions:
// 1. Open a part document that has an ImportedCurve1 feature.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Selects the ImportedCurve1 feature.
// 2. Gets the number of curves in ImportedCurve1.
// 3. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ImportedCurveFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            ImportedCurveFeatureData swImportedCurveFeatureData = default(ImportedCurveFeatureData);
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;

            //Select ImportedCurve1 and get the number of curves in the feature
            status = swModelDocExt.SelectByID2("ImportedCurve1", "REFERENCECURVES", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swImportedCurveFeatureData = (ImportedCurveFeatureData)swFeature.GetDefinition();
            Debug.Print("Number of curves: " + swImportedCurveFeatureData.GetCurveCount());
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Get Imported Curve Feature Data Example (VBA)

This example shows how to get the number of curves in an imported curve feature.

```
'-----  
' Preconditions:  
' 1. Open a part document that has an ImportedCurve1 feature.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Selects the ImportedCurve1 feature.  
' 2. Gets the number of curves in ImportedCurve1.  
' 3. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swFeature As SldWorks.Feature  
Dim swImportedCurveFeatureData As SldWorks.ImportedCurveFeatureData  
Dim status As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swSelectionMgr = swModel.SelectionManager  
  
    'Select ImportedCurve1 and get the number of curves in the feature  
    status = swModelDocExt.SelectByID("ImportedCurve1", "REFERENCECURVES", 0, 0, 0, False, 0, Nothing, 0)  
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)  
    Set swImportedCurveFeatureData = swFeature.GetDefinition  
    Debug.Print "Number of curves: " & swImportedCurveFeatureData.GetCurveCount  
  
End Sub
```

Get Imported Curve Feature Data Example (VB.NET)

This example shows how to get the number of curves in an imported curve feature.

```

-----
Preconditions:
1. Open a part document that has an ImportedCurve1 feature.
2. Open the Immediate window.
-----
Postconditions:
1. Selects the ImportedCurve1 feature.
2. Gets the number of curves in ImportedCurve1.
3. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swImportedCurveFeatureData As ImportedCurveFeatureData
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelectionMgr = swModel.SelectionManager

        'Select ImportedCurve1 and get the number of curves in the feature
        status = swModelDocExt.SelectByID("ImportedCurve1", "REFERENCECURVES", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swImportedCurveFeatureData = swFeature.GetDefinition
        Debug.Print("Number of curves: " & swImportedCurveFeatureData.GetCurveCount)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Info on Plane/Axis Example (C++ COM)

This example shows how to get parameter information for the currently selected reference plane or reference axis.

```
void GetRefObjectInfo()
{
    LPMODELDOC m_ModelDoc = NULL;
    HRESULT hres = NOERROR;

    // Retrieve IModelDoc pointer
    hres = UserApp->getSWApp()->get_IActiveDoc( &m_ModelDoc );
    if( m_ModelDoc == NULL )
        return;

    long type;

    // Helper function
    IFeature* m_Feature = GetSelectedFeature( m_ModelDoc, &type );

    if( m_Feature != NULL )
    {
        BSTR bFeatName;

        // Get feature name
        hres = m_Feature->get_Name( &bFeatName );
        BSTR bTypeName;

        // Get the feature type
        hres = m_Feature->GetTypeName( &bTypeName );
        CString featName( bFeatName );
        CString typeName( bTypeName );
        CString message;

        // Display message to user
        message.Format( _T("Type: %s\nName: %s"), typeName, featName );

        SysFreeString(bFeatName);
        SysFreeString(bTypeName);

        // If the selected feature is a Reference Plane
        if (type == swSelDATUMPLANES)
        {
            LPUNKNOWN iUnk = NULL;

            // Get the RefPlane interface
            hres = m_Feature->IGetSpecificFeature( &iUnk );

            IRefPlane* refPlane = NULL;
            hres = iUnk->QueryInterface( IID_IRefPlane, (LPVOID*)&refPlane );
            double values[9];

            // Get Ref Plane parameters
            hres = refPlane->IGetRefPlaneParams( values );

            CString message2;          // Display message to user
            message2.Format( _T("RefPlaneParams\nOrigin:%f, %f, %f\n X-Dir:%f, %f, %f\nNormal:%f, %f, %f"), values[0], values[1], values[2], values[3], values[4], values[5], values[6], values[7], values[8] );
            AfxMessageBox( message2 );

            refPlane->Release();
        }
        else if (type == swSelDATUMAXES)
        // If the selected feature is a Reference Axis
        {
            LPUNKNOWN iUnk = NULL;

            // Get the RefAxis interface
```

```

hres = m_Feature->IGetSpecificFeature( &iUnk );

IRefAxis* refAxis = NULL;
hres = iUnk->QueryInterface( IID_IRefAxis, (LPVOID*)&refAxis);
double values[6];

// Get Ref Axis parameters
hres = refAxis->IGetRefAxisParams( values );

// Display message to user
CString message2;
message2.Format( _T("RefPlaneParams\nStartPt:%f, %f, %f\nEndPt:%f, %f, %f"), values[0], values[1], values[2], values[3], values[4], values[5] );
AfxMessageBox( message2 );

refAxis->Release();
}

// Display message to user
AfxMessageBox( message );

}

m_ModelDoc->Release();
m_Feature->Release();

}

```

Get Intersect Feature Data Example (C#)

This example shows how to create an intersect feature and get its data.

```
//-----
// Preconditions:
// 1. Open a part that contains the following intersecting features:
//    * Boss-Extrude1
//    * Boss-Extrude2
//    * Boss-Extrude3
//    * Boss-Extrude4
//    * Surface-Extrude1
// 2. Add reference, Microsoft.VisualBasic, to the project.
// 3. Open an Immediate window.
// 4. Multi-select Surface-Extrude1, Boss-Extrude3, and Boss-Extrude4 in the
//    FeatureManager design tree and press F5.
//
// Postconditions:
// 1. When the macro stops, inspect the blue intersect regions.
// 2. Press F5.
// 3. Inspect the Immediate window.
// 4. Right-click Intersect1 in the FeatureManager design tree and click
//    Roll Forward.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace ModifyIntersect_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        Feature swFeat;
        FeatureManager swFeatMgr;
        IntersectFeatureData featData;

        int intStatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swFeatMgr = swModel.FeatureManager;

            object[] vResultingBodies = null;

            vResultingBodies = (object[])swFeatMgr.PreIntersect(false);
            //Do not cap planar surface openings of intersect feature
            swModel.ClearSelection2(true);

            int i = 0;
            Body2 swBody = default(Body2);

            Debug.Print("");
            //Color the intersect regions blue
            for (i = 0; i < vResultingBodies.Length; i++)
            {
                swBody = (Body2)vResultingBodies[i];
                Debug.Print("Intersect region " + i + 1 + " is a temporary body? " + swBody.IsTemporaryBody());
                intStatus = swBody.Display3(swModel, 16711680, (int)swTempBodySelectOptions_e.swTempBodySelectOptionNone);
                Debug.Print("Intersect region " + i + 1 + " is displayed (0 = yes)? " + intStatus);
            }

            System.Diagnostics.Debugger.Break();
            //Observe the intersect regions

            object intToExclude = null;
            bool[] boolArr = new bool[4];
            boolArr[0] = false;
            boolArr[1] = true;
            // Exclude region, vResultingBodies(2), from the intersect feature
            boolArr[2] = false;
            boolArr[3] = false;
            intToExclude = boolArr;
            swFeat = swFeatMgr.PostIntersect(intToExclude, true, false);

            Debug.Print("Feature name = " + swFeat.Name);
            featData = (IntersectFeatureData)swFeat.GetDefinition();

            Debug.Print("Merge touching regions into one body? " + featData.Merge);
        }
    }
}
```

```
        Debug.Print("Consume surfaces? " + featData.Consume);
        Debug.Print("Cap planar openings on surfaces? " + featData.CapPlanar);

        Debug.Print("Number of solids, surfaces, or planes used to create the intersect feature: " +
featData.GetIntersectionsToolsCount());
        Debug.Print("Number of intersect regions: " + featData.GetIntersectionsCount());
    }

    public SldWorks swApp;
}
}
```

Get Intersect Feature Data Example (VBA)

This example shows how to create an intersect feature and get its data.

```

'-----
' Preconditions:
' 1. Open a part that contains the following intersecting features:
'    * Boss-Extrude1
'    * Boss-Extrude2
'    * Boss-Extrude3
'    * Boss-Extrude4
'    * Surface-Extrude1
' 2. Open an Immediate window.
' 3. Multi-select Surface-Extrude1, Boss-Extrude3, and Boss-Extrude4 in the
'    FeatureManager design tree and press F5.
'
' Postconditions:
' 1. When the macro stops, hide Boss-Extrude3 and Boss-Extrude4 in the
'    FeatureManager design tree.
' 2. Press F5.
' 3. When the macro stops, inspect the blue intersect regions.
' 4. Press F5.
' 5. Inspect the Immediate window.
' 6. Right-click Intersect1 in the FeatureManager design tree and click
'    Roll Forward.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeat As SldWorks.Feature
Dim swFeatMgr As SldWorks.FeatureManager
Dim featData As SldWorks.IntersectFeatureData
Dim intStatus As Long

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager

    Dim vResultingBodies As Variant

    vResultingBodies = swFeatMgr.PreIntersect(False) 'Do not cap planar surface openings of intersect feature
    swModel.ClearSelection2 True

    Dim i As Integer
    Dim swBody As Body2

    Stop 'Hide Boss-Extrude4 and Boss-Extrude3 to expose the intersect regions

    Debug.Print ""
    'Color the intersect regions blue
    For i = 0 To UBound(vResultingBodies)
        Set swBody = vResultingBodies(i)
        Debug.Print "Intersect region " & i + 1 & " is a temporary body? " & swBody.IsTemporaryBody
        intStatus = swBody.Display3(swModel, 16711680, swTempBodySelectOptionNone)
        Debug.Print "Intersect region " & i + 1 & " is displayed (0 = yes)? " & intStatus
    Next

    Stop 'Observe the intersect regions

    Dim intToExclude As Variant
    Dim boolArr(3) As Boolean
    boolArr(0) = False
    boolArr(1) = True 'Exclude region, vResultingBodies(2), from the intersect feature
    boolArr(2) = False
    boolArr(3) = False
    intToExclude = boolArr
    Set swFeat = swFeatMgr.PostIntersect(intToExclude, True, False)

    Debug.Print "Feature name = " & swFeat.Name
    Set featData = swFeat.GetDefinition

    Debug.Print "Merge touching regions into one body? " & featData.Merge
    Debug.Print "Consume surfaces? " & featData.Consume
    Debug.Print "Cap planar openings on surfaces? " & featData.CapPlanar

    Debug.Print "Number of solids, surfaces, or planes used to create the intersect feature: " &
    featData.GetIntersectionsToolsCount
    Debug.Print "Number of intersect regions: " & featData.GetIntersectionsCount

End Sub

```

Get Intersect Feature Data Example (VB.NET)

This example shows how to create an intersect feature and get its data.

```

'-----
' Preconditions:
' 1. Open a part that contains the following intersecting features:
'    * Boss-Extrude1
'    * Boss-Extrude2
'    * Boss-Extrude3
'    * Boss-Extrude4
'    * Surface-Extrude1
' 2. Open an Immediate window.
' 3. Multi-select Surface-Extrude1, Boss-Extrude3, and Boss-Extrude4 in the
'    FeatureManager design tree and press F5.
'
' Postconditions:
' 1. When the macro stops, inspect the blue intersect regions.
' 2. Press F5.
' 3. Inspect the Immediate window.
' 4. Right-click Intersect1 in the FeatureManager design tree and click
'    Roll Forward.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim swModel As ModelDoc2
    Dim swFeat As Feature
    Dim swFeatMgr As FeatureManager
    Dim featData As IntersectFeatureData
    Dim intStatus As Integer

    Sub main()

        swModel = swApp.ActiveDoc
        swFeatMgr = swModel.FeatureManager

        Dim vResultingBodies As Object

        vResultingBodies = swFeatMgr.PreIntersect(False) 'Do not cap planar surface openings of intersect feature
        swModel.ClearSelection2(True)

        Dim i As Integer
        Dim swBody As Body2

        Debug.Print("")
        'Color the intersect regions blue
        For i = 0 To UBound(vResultingBodies)
            swBody = vResultingBodies(i)
            Debug.Print("Intersect region " & i + 1 & " is a temporary body? " & swBody.IsTemporaryBody)
            intStatus = swBody.Display3(swModel, 16711680, swTempBodySelectOptions_e.swTempBodySelectOptionNone)
            Debug.Print("Intersect region " & i + 1 & " is displayed (0 = yes)? " & intStatus)
        Next

        Stop 'Observe the intersect regions

        Dim intToExclude As Object
        Dim boolArr(3) As Boolean
        boolArr(0) = False
        boolArr(1) = True ' Exclude region, vResultingBodies(2), from the intersect feature
        boolArr(2) = False
        boolArr(3) = False
        intToExclude = boolArr
        swFeat = swFeatMgr.PostIntersect(intToExclude, True, False)

        Debug.Print("Feature name = " & swFeat.Name)
        featData = swFeat.GetDefinition

        Debug.Print("Merge touching regions into one body? " & featData.Merge)
        Debug.Print("Consume surfaces? " & featData.Consume)
        Debug.Print("Cap planar openings on surfaces? " & featData.CapPlanar)

        Debug.Print("Number of solids, surfaces, or planes used to create the intersect feature: " &
            featData.GetIntersectionsToolsCount)
        Debug.Print("Number of intersect regions: " & featData.GetIntersectionsCount)

    End Sub

    Public swApp As SldWorks

```

End Class

-

Get Intersecting Face and Edge Example (VBA)

This example shows how to get the intersection of a face and an edge.

NOTE: A face and an edge intersect in a series of points. This example shows how to use some of the geometry- and topology-related methods to get the intersection points.

```

'-----
' Preconditions:
' 1. Open a part or a fully resolved assembly.
' 2. Select an intersecting face and edge, in that
'    order, in the graphics area.
'
' Postconditions:
' 1. Inserts a 3D sketch with sketch points located
'    at the intersection of the face and edge.
' 2. Examine the graphics area and FeatureManager design
'    tree.
'-----
Option Explicit

Sub CreatePoints(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vPtArray As Variant)
    Dim swSketchPt As SldWorks.SketchPoint
    Dim bRet As Boolean
    Dim i As Long

    swModel.SetAddToDB True
    swModel.Insert3DSketch2 False

    For i = 0 To UBound(vPtArray) Step 3
        Set swSketchPt = swModel.CreatePoint2(vPtArray(i + 0), vPtArray(i + 1), vPtArray(i + 2))
    Next i

    swModel.SetAddToDB False
    swModel.Insert3DSketch2 True

    bRet = swModel.EditRebuild3
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFace As SldWorks.Face2
    Dim swSurf As SldWorks.Surface
    Dim swEdge As SldWorks.Edge
    Dim swCurve As SldWorks.Curve
    Dim vPointArray As Variant
    Dim vTArray As Variant
    Dim vUArray As Variant
    Dim vCurveParam As Variant
    Dim vCurveBounds As Variant
    Dim nCurveBounds(0 To 5) As Double
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    Set swEdge = swSelMgr.GetSelectedObject6(2, -1)
    Set swSurf = swFace.GetSurface
    Set swCurve = swEdge.GetCurve

    vCurveParam = swEdge.GetCurveParams2

    For i = 0 To 5
        nCurveBounds(i) = vCurveParam(i)
    Next i
    vCurveBounds = nCurveBounds

    bRet = swSurf.IntersectCurve(swCurve, (vCurveBounds), vPointArray, vTArray, vUArray)

    CreatePoints swApp, swModel, vPointArray
End Sub

```

Get Intersecting Faces Example (VBA)

This example shows how to get the intersection of two faces.

NOTE: This example uses some geometry- and topology-related methods and properties; e.g., `ISurface::IntersectSurface`, to get intersecting surfaces.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\cstick.sldprt.
' 2. Select the face of the finger holder that intersects
'    the curved face at the base of the candlestick
'    and the curved face at the base of the candlestick holder.
'
' Postconditions:
' 1. Inserts a 3D sketch with a series of line segments
'    approximating the intersection for each intersection curve.
' 2. Examine the graphics area and FeatureManager design tree.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Sub CreateTessCurve(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swTrimCurve As SldWorks.Curve)
    Dim nChordTol As Double
    Dim nLengthTol As Double
    Dim nStartParam As Double
    Dim nEndParam As Double
    Dim bIsClosed As Boolean
    Dim bIsPeriodic As Boolean
    Dim vStartPt As Variant
    Dim vEndPt As Variant
    Dim vTessPts As Variant
    Dim swSketchSeg As SldWorks.SketchSegment
    Dim bRet As Boolean
    Dim i As Long

    ' Not needed because curve is a trimmed curve
    ' Could pass in trim points as parameters
    bRet = swTrimCurve.GetEndParams(nStartParam, nEndParam, bIsClosed, bIsPeriodic)

    vStartPt = swTrimCurve.Evaluate(nStartParam)
    vEndPt = swTrimCurve.Evaluate(nEndParam)

    nChordTol = 0.001
    nLengthTol = 0.001
    vTessPts = swTrimCurve.GetTessPts(nChordTol, nLengthTol, (vStartPt), (vEndPt))

    swModel.SetAddToDB True
    swModel.Insert3DSketch2 False

    ' Disable Visual Basic range checking because tessellation points
    ' might not be a multiple of 6
    On Error Resume Next
    For i = 0 To UBound(vTessPts) Step 6
        Set swSketchSeg = swModel.CreateLine2(vTessPts(i + 0), vTessPts(i + 1), vTessPts(i + 2), vTessPts(i + 3), vTessPts(i + 4), vTessPts(i + 5))
    Next i
    On Error GoTo 0

    swModel.SetAddToDB False
    swModel.Insert3DSketch2 True
End Sub

Function CreateTrimmedCurve(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSurf As SldWorks.Surface, swCurve As SldWorks.Curve) As SldWorks.Curve
    Dim nStartParam As Double
    Dim nEndParam As Double
    Dim bIsClosed As Boolean
    Dim bIsPeriodic As Boolean
    Dim vStartPt As Variant
    Dim vEndPt As Variant
    Dim nCurveBounds(0 To 5) As Double
    Dim vCurveBounds As Variant
    Dim vPointArray As Variant
    Dim vTArray As Variant
    Dim vUArray As Variant
    Dim bRet As Boolean

    bRet = swCurve.GetEndParams(nStartParam, nEndParam, bIsClosed, bIsPeriodic)

    vStartPt = swCurve.Evaluate(nStartParam)
    vEndPt = swCurve.Evaluate(nEndParam)
    nCurveBounds(0) = vStartPt(0)
    nCurveBounds(1) = vStartPt(1)
    nCurveBounds(2) = vStartPt(2)
    nCurveBounds(3) = vEndPt(0)
    nCurveBounds(4) = vEndPt(1)
    nCurveBounds(5) = vEndPt(2)
    vCurveBounds = nCurveBounds
    bRet = swSurf.IntersectCurve(swCurve, (vCurveBounds), vPointArray, vTArray, vUArray)

    Set CreateTrimmedCurve = swCurve.CreateTrimmedCurve(vPointArray(0), vPointArray(1), vPointArray(2), vPointArray(3), vPointArray(4), vPointArray(5))
End Function

Sub main()

```

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFace1 As SldWorks.Face2
Dim swFace2 As SldWorks.Face2
Dim swSurf1 As SldWorks.Surface
Dim swSurf2 As SldWorks.Surface
Dim swCurve1 As SldWorks.Curve
Dim swCurve2 As SldWorks.Curve
Dim vCurveArray As Variant
Dim vBoundArray As Variant
Dim swIntCurve As SldWorks.Curve
Dim swTrimCurve As SldWorks.Curve
Dim nLength As Double
Dim i As Long
Dim bRet As Boolean

Set swApp = CreateObject("SldWorks.Application")
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swFace1 = swSelMgr.GetSelectedObject6(1, -1)
Set swFace2 = swSelMgr.GetSelectedObject6(2, -1)
Set swSurf1 = swFace1.GetSurface
Set swSurf2 = swFace2.GetSurface

bRet = swSurf1.IntersectSurface(swSurf2, vCurveArray, vBoundArray)

For i = 0 To UBound(vCurveArray)
    Set swIntCurve = vCurveArray(i)
    ' Curve could be infinite (that is, a straight line)
    If swIntCurve.IsTrimmedCurve Then
        nLength = swIntCurve.GetLength2(vBoundArray(2 * i), vBoundArray(2 * i + 1))
        Debug.Print "Curve(" & i & ") = " & nLength * 1000# & " mm"
        Set swTrimCurve = swIntCurve
    Else
        ' Create a trimmed curve by re-intersecting
        ' intersection curve with surface
        Set swTrimCurve = CreateTrimmedCurve(swApp, swModel, swSurf1, swIntCurve)
    End If
    CreateTessCurve swApp, swModel, swTrimCurve
Next i

bRet = swModel.EditRebuild3

End Sub

```

Get Knit Surface Data Example (VBA)

This example shows how to get a knit surface's data.

```

'-----
' Preconditions:
' 1. Verify that the specified part template
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates two surfaces.
' 3. Creates a knit surface feature using the two
'    selected surfaces.
' 4. Examine the graphics area, FeatureManager design
'    tree, and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeatureManager As SldWorks.FeatureManager
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swSelData As SldWorks.SelectData
Dim swFeature As SldWorks.Feature
Dim swSurfaceKnitFeature As SldWorks.SurfaceKnitFeatureData
Dim vEnt As Variant
Dim swEnt As SldWorks.Entity
Dim swFace As SldWorks.Face2
Dim swSeedFace As SldWorks.Face2
Dim swSurf As SldWorks.Surface
Dim swSurfRadFeat As SldWorks.Feature
Dim i As Long
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks

    'Open new part document and create two surfaces
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    Set swSketchManager = swModel.SketchManager
    Set swFeatureManager = swModel.FeatureManager
    Set swSelectionManager = swModel.SelectionManager
    Set swSelData = swSelectionManager.CreateSelectData
    boolstatus = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchManager.CreateEllipse(-4.15374666666667E-02, 0, 0, 5.34585333333333E-02, 0, 0, -4.15374666666667E-02, 2.08618666666667E-02, 0)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
    boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    swFeatureManager.FeatureExtruRefSurface2 True, False, 0, 0, 0.05, 0.01, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, False, False, False
    swSelectionManager.EnableContourSelection = False
    boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, True, 1, Nothing, 0)
    boolstatus = swModel.InsertPlanarRefSurface()
    swModel.ClearSelection2 True

    ' Select both surfaces and create surface knit feature
    boolstatus = swModelDocExt.SelectByID("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, False, 1, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("Surface-Plane1", "SURFACEBODY", 0, 0, 0, True, 1, Nothing, 0)
    Set swFeature = swFeatureManager.InsertSewRefSurface(True, False, False, 0.0001, 0.0001)

    ' Get some surface knit feature data
    Set swSurfaceKnitFeature = swFeature.GetDefinition

    ' Roll back to get to geometric entities
    boolstatus = swSurfaceKnitFeature.AccessSelections(swModel, Nothing)

    Set swSeedFace = swSurfaceKnitFeature.SeedFace
    If Not swSeedFace Is Nothing Then
        swModel.ClearSelection2 True
        Set swEnt = swSeedFace
        Debug.Print "Seed entity type: " & swEnt.GetType
        boolstatus = swEnt.Select4(True, swSelData)
    End If

    ' Get knit surface data
    boolstatus = False
    swModel.ClearSelection2 True
    vEnt = swSurfaceKnitFeature.Entities
    For i = 0 To UBound(vEnt)
        Set swEnt = vEnt(i)
        Set swSurfRadFeat = vEnt(i)
        Debug.Print "Entity type (" & i & "): " & swEnt.GetType
        If swSelectType_e.swSelFACES = swEnt.GetType Then

```

```
        boolstatus = swEnt.Select4(True, swSelData):
    Else
        ' Although not a surface-radiate feature, you cannot select
        ' a surface-radiate feature through IEntity interface, so select
        ' through IFeature interface
        Debug.Print " Feature type: " & swSurfRadFeat.GetTypeName
        boolstatus = swSurfRadFeat.Select2(True, 0)
    End If
Next i

' Apply any changes
boolstatus = swFeature.ModifyDefinition(swSurfaceKnitFeature, swModel, Nothing)

End Sub
```

Get Labels of Datum Origin Example (C#)

This example shows how to insert a hole table annotation and get some of its properties.

```
//-----
// Preconditions:
// 1. Verify that the specified files exist:
//   * part document
//   * drawing document template
//   * hole table template
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Creates a drawing of the part document.
// 3. Inserts a hole table annotation at the specified vertex.
// 4. Prints to the Immediate window:
//   * the hole table's datum origin x and y labels
//   * whether to display hole sizes using ANSI inch letters
//     and drill numbers
// 5. Examine the Immediate window and drawing.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetLabels_CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {

            ModelDoc2 swModel;
            DrawingDoc swDrawingDoc;
            View swView;
            ModelView swModelView;
            ModelDocExtension swModelDocExt;
            SelectionMgr swSelectionMgr;
            HoleTableAnnotation swHoleTableAnnotation;
            HoleTable swHoleTable;
            DatumOrigin swDatumOrigin;
            string fileName;
            string drawingDocTemplate;
            string holeTableTemplate;
            bool status;
            int errors = 0;
            int warnings = 0;

            //Open part document and create drawing document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\tolanalyst\\minimum_clearance\\top_plate.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions
            drawingDocTemplate = "C:\\ProgramData\\SolidWorks\\SolidWorks 2016\\templates\\Drawing.drwdot";
            swDrawingDoc = (DrawingDoc)swApp.NewDocument(drawingDocTemplate, (int)swDwgPaperSizes_e.swDwgPaperBsize, 0.2794, 0.4318,
            swView = (View)swDrawingDoc.CreateDrawViewFromModelView3(fileName, "Front", 0.193046827497194, 0.155595183164983, 0);

            //Insert hole table annotation
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDrawingDoc = (DrawingDoc)swModel;
            swModelView = (ModelView)swModel.ActiveView;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swDrawingDoc.ActivateView("Drawing View1");
            status = swModelDocExt.SelectByID2("", "VERTEX", 0.216278249046016, 0.198389907070707, 0.008000000000000382, false, 0, nul
            status = swModelDocExt.SelectByID2("", "FACE", 0.228097744219978, 0.151927063973064, 0.004000000000000191, false, 0, null,
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("", "VERTEX", 0.216278249046016, 0.198389907070707, 0.008000000000000382, true, 1, null
            status = swModelDocExt.SelectByID2("", "FACE", 0.228097744219978, 0.151927063973064, 0.004000000000000191, true, 2, null,
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swView = (View)swSelectionMgr.GetSelectedObjectsDrawingView2(1, -1);
            holeTableTemplate = "C:\\Program Files\\SolidWorks Corp\\SolidWorks\\lang\\english\\standard hole table--
letters.sldholtbt";
            swHoleTableAnnotation = (HoleTableAnnotation)swView.InsertHoleTable2(false, 0.0540658670033671, 0.253819263748597, 1, "
            swModel.ClearSelection2(true);

            //Get hole table and its datum origin
            swHoleTable = (HoleTable)swHoleTableAnnotation.HoleTable;
            swDatumOrigin = (DatumOrigin)swHoleTable.DatumOrigin;

            //Get datum origin's x and y labels
            Debug.Print("Datum origin:");
            Debug.Print(" X label: " + swDatumOrigin.XLabel);
```

```
        Debug.Print(" Y label: " + swDatumOrigin.YLabel);

        //Get whether to display hole sizes using ANSI inch letters and drill numbers
        Debug.Print("Display ANSI inch letter and drill number sizes in hole
table? " + swHoleTable.ShowANSIInchLetterNumberDrillSizes);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Get Labels of Datum Origin Example (VBA)

This example shows how to insert a hole table annotation and get some of its properties.

```

'-----
' Preconditions:
' 1. Verify that the specified files exist:
'    * part document
'    * drawing document template
'    * hole table template
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Creates a drawing of the part document.
' 3. Inserts a hole table annotation at the specified vertex.
' 4. Prints to the Immediate window:
'    * the hole table's datum origin x and y labels
'    * whether to display hole sizes using ANSI inch letters
'      and drill numbers
' 5. Examine the Immediate window and drawing.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim swModelView As SldWorks.ModelView
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swHoleTableAnnotation As SldWorks.HoleTableAnnotation
Dim swHoleTable As SldWorks.HoleTable
Dim swDatumOrigin As SldWorks.DatumOrigin
Dim fileName As String
Dim drawingDocTemplate As String
Dim holeTableTemplate As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Sub main()
    Set swApp = Application.SldWorks
    'Open part document and create drawing document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS\
2018\samples\tutorial\tolanalyst\minimum_clearance\top_plate.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
warnings)
    drawingDocTemplate = "C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Drawing.drwdot"
    Set swDrawingDoc = swApp.NewDocument(drawingDocTemplate, swDwgPaperSizes_e.swDwgPaperBsize, 0.2794, 0.4318)
    Set swView = swDrawingDoc.CreateDrawViewFromModelView3(fileName, "Front", 0.193046827497194, 0.155595183164983, 0)
    'Insert hole table annotation
    Set swModel = swApp.ActiveDoc
    Set swDrawingDoc = swModel
    Set swModelView = swModel.ActiveView
    Set swModelDocExt = swModel.Extension
    status = swDrawingDoc.ActivateView("Drawing View1")
    status = swModelDocExt.SelectByID("", "VERTEX", 0.216278249046016, 0.198389907070707, 8.0000000000382E-03, False, 0, Nothing,
0)
    status = swModelDocExt.SelectByID("", "FACE", 0.228097744219978, 0.151927063973064, 4.0000000000191E-03, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("", "VERTEX", 0.216278249046016, 0.198389907070707, 8.0000000000382E-03, True, 1, Nothing,
0)
    status = swModelDocExt.SelectByID("", "FACE", 0.228097744219978, 0.151927063973064, 4.0000000000191E-03, True, 2, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swView = swSelectionMgr.GetSelectedObjectsDrawingView2(1, -1)
    holeTableTemplate = "C:\Program Files\SolidWorks Corp\SolidWorks\lang\english\standard hole table--letters.sldholtbt"
    Set swHoleTableAnnotation = swView.InsertHoleTable2(False, 5.40658670033671E-02, 0.253819263748597, 1, "A", holeTableTemplate)
    swModel.ClearSelection2 True
    'Get hole table and its datum origin
    Set swHoleTable = swHoleTableAnnotation.HoleTable
    Set swDatumOrigin = swHoleTable.DatumOrigin
    'Get datum origin's x and y labels
    Debug.Print "Datum origin:"
    Debug.Print (" X label: " & swDatumOrigin.XLabel)
    Debug.Print (" Y label: " & swDatumOrigin.YLabel)
    'Get whether to display hole sizes using ANSI inch letters and drill numbers
    Debug.Print "Display ANSI inch letter and drill number sizes in hole table? " & swHoleTable.ShowANSIInchLetterNumberDrillSizes
End Sub

```

Get Labels of Datum Origin Example (VB.NET)

This example shows how to insert a hole table annotation and get some of its properties.

```

'-----
' Preconditions:
' 1. Verify that the specified files exist:
'   * part document
'   * drawing document template
'   * hole table template
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Creates a drawing of the part document.
' 3. Inserts a hole table annotation at the specified vertex.
' 4. Prints to the Immediate window:
'   * the hole table's datum origin x and y labels
'   * whether to display hole sizes using ANSI inch letters
'     and drill numbers
' 5. Examine the Immediate window and drawing.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swDrawingDoc As DrawingDoc
    Dim swView As View
    Dim swModelView As ModelView
    Dim swModelDocExt As ModelDocExtension
    Dim swSelectionMgr As SelectionMgr
    Dim swHoleTableAnnotation As HoleTableAnnotation
    Dim swHoleTable As HoleTable
    Dim swDatumOrigin As DatumOrigin
    Dim fileName As String
    Dim drawingDocTemplate As String
    Dim holeTableTemplate As String
    Dim status As Boolean
    Dim errors As Integer
    Dim warnings As Integer

    Sub main()

        'Open part document and create drawing document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\tolanalyst\minimum_clearance\top_plate.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warr
        drawingDocTemplate = "C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Drawing.drwdot"
        swDrawingDoc = swApp.NewDocument(drawingDocTemplate, swDwgPaperSizes_e.swDwgPaperBsize, 0.2794, 0.4318)
        swView = swDrawingDoc.CreateDrawViewFromModelView3(fileName, "**Front", 0.193046827497194, 0.155595183164983, 0)

        'Insert hole table annotation
        swModel = swApp.ActiveDoc
        swDrawingDoc = swModel
        swModelView = swModel.ActiveView
        swModelDocExt = swModel.Extension
        status = swDrawingDoc.ActivateView("Drawing View1")
        status = swModelDocExt.SelectByID2("", "VERTEX", 0.216278249046016, 0.198389907070707, 0.008000000000000382, False, 0, Nothing,
        status = swModelDocExt.SelectByID2("", "FACE", 0.228097744219978, 0.151927063973064, 0.004000000000000191, False, 0, Nothing,
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("", "VERTEX", 0.216278249046016, 0.198389907070707, 0.008000000000000382, True, 1, Nothing,
        status = swModelDocExt.SelectByID2("", "FACE", 0.228097744219978, 0.151927063973064, 0.004000000000000191, True, 2, Nothing,
        swSelectionMgr = swModel.SelectionManager
        swView = swSelectionMgr.GetSelectedObjectsDrawingView2(1, -1)
        holeTableTemplate = "C:\Program Files\SolidWorks Corp\SolidWorks\lang\english\standard hole table--letters.sldholtbt"
        swHoleTableAnnotation = swView.InsertHoleTable2(False, 0.0540658670033671, 0.253819263748597, 1, "A", holeTableTemplate)
        swModel.ClearSelection2(True)

        'Get hole table and its datum origin
        swHoleTable = swHoleTableAnnotation.HoleTable
        swDatumOrigin = swHoleTable.DatumOrigin

        'Get datum origin's x and y labels
        Debug.Print("Datum origin:")
        Debug.Print(" X label: " & swDatumOrigin.XLabel)
        Debug.Print(" Y label: " & swDatumOrigin.YLabel)

        'Get whether to display hole sizes using ANSI inch letters and drill numbers
        Debug.Print("Display ANSI inch letter and drill number
sizes in the hole table? " & swHoleTable.ShowANSIInchLetterNumberDrillSizes)

    End Sub

```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```